

TÀI LIỆU KHẢO SÁT CHUYÊN SÂU

KIẾN TRÚC MINI-TRANSFORMER DỊCH MÁY VIỆT - ANH

Đề tài: Phát triển Hệ thống dịch máy song ngữ seq2seq ứng dụng cơ chế Attention tự xây dựng từ con số không (From Scratch)

Tập dữ liệu: IWSLT 2015 English-Vietnamese (Tổng: 135.853 câu; Train: 133.317 (~98,13%), Val: 1.268 (~0,93%), Test: 1.268 (~0,93%))

Chi tiết giải thuật: Multi-Head Attention, Pre-LN LayerNorm, Shared BPE, Noam Scheduler

Chiến lược tối ưu: Multi-GPU DataParallel, FP16 AMP, Gradient Accumulation, Bucket Batching

Tác giả: Nhóm Nghiên cứu Khoa học & Xử lý Ngôn ngữ Tự nhiên (NCKH Team)

Ngày thực hiện: 01 tháng 06 năm 2026

TÀI LIỆU KHẢO SÁT CHUYÊN SÂU KIẾN TRÚC MINI-TRANSFORMER DỊCH MÁY VIỆT - ANH

PHẦN I: TỔNG QUAN KIẾN TRÚC & KHỞI TẠO

1. Phân tích Sơ đồ Khối và Mã nguồn thực tế

Trực quan hóa Sơ đồ Tổng quan Kiến trúc và Các khối Layers:

Phân tích & So sánh Kiến trúc	Sơ đồ Luồng Tổng quan
Khi so sánh sơ đồ luồng dữ liệu tổng quát với kiến trúc Transformer gốc trong bài báo, xét về mặt nguyên lý cốt lõi, không có sự khác biệt. Tuy nhiên, có hai điểm khác biệt cụ thể về thông số và cách minh họa trong dự án: 1. Số lượng tầng (Layers): - Sơ đồ: Vẽ 3 Layers để tối ưu hóa hiển thị. - Mã nguồn: Dùng cấu hình chuẩn 6 Layers, __CODE_BLOCK_0__, __CODE_BLOCK_1__ (Base). 2. Bài toán áp dụng: - Sơ đồ: Dịch máy Việt - Anh với dữ liệu mẫu cụ thể ("chúng tôi học sâu" -> "we learn"). - Bài báo: Tổng quát, kiểm thử trên tiếng Anh - Đức/Pháp.	* Sơ đồ tổng quan kiến trúc Mini-Transformer Việt - Anh
Cấu trúc chi tiết Encoder Layer	Cấu trúc chi tiết Decoder Layer
* Sơ đồ khối Encoder Layer	* Sơ đồ khối Decoder Layer

2. So sánh Kiến trúc so với Bài báo gốc (Attention Is All You Need)

2.1. Các Điểm Khác Biệt Cốt Lõi So Với Bài Báo Gốc

Đối với kiến trúc Transformer trong dự án hiện tại của chúng ta so với phiên bản gốc trong bài báo "Attention Is All You Need" (2017), có một số điểm cải tiến và tùy chỉnh quan trọng như sau:

Đối với kiến trúc Transformer trong codebase hiện tại của chúng ta so với phiên bản gốc trong bài báo "Attention Is All You Need" (2017), có một số điểm cải tiến và tùy chỉnh quan trọng như sau:

2.2. Kiến trúc Layer Normalization (Pre-LN vs Post-LN)

- Mô hình của chúng ta (Pre-LN): Layer Normalization được áp dụng trước khi đi vào các khối con (Self-Attention hoặc Feed-Forward) rồi mới cộng kết nối tắt (residual), thể hiện rõ trong mã nguồn lớp EncoderLayer và DecoderLayer:

$$x = x + \text{Sublayer}(\text{LayerNorm}(x))$$

- Bài báo gốc (Post-LN): Áp dụng Layer Normalization sau khi cộng kết nối tắt:

$$x = \text{LayerNorm}(x + \text{Sublayer}(x))$$

- Ý nghĩa: Thiết kế Pre-LN (được dùng trong các mô hình hiện đại như GPT, LLaMA) giúp luồng gradient truyền đi ổn định hơn rất nhiều, tránh hiện tượng bùng nổ hoặc triệt tiêu gradient ở các tầng sâu, giúp huấn luyện dễ dàng hơn.

2.3. Kích thước và Siêu tham số (Hyperparameters)

Mô hình của chúng ta được thiết kế ở dạng Mini-Transformer để tối ưu hóa tài nguyên phần cứng cá nhân và tập dữ liệu dịch Việt - Anh nhỏ/vừa:

- Mô hình của chúng ta:
- Số lớp Encoder & Decoder (___MATH_BLOCK_0___): 3 layers (xác định trong Transformer).
- Số chiều vector biểu diễn (___MATH_BLOCK_0___): 256.
- Số chiều lớp ẩn Feed-Forward (___MATH_BLOCK_0___): 512 (chỉ gấp 2 lần ___MATH_BLOCK_1___).
- Bài báo gốc (Bản Base):
- Số lớp Encoder & Decoder (___MATH_BLOCK_0___): 6 layers.
- Số chiều vector biểu diễn (___MATH_BLOCK_0___): 512.
- Số chiều lớp ẩn Feed-Forward (___MATH_BLOCK_0___): 2048 (gấp 4 lần ___MATH_BLOCK_1___).
- Khởi tạo trọng số (Weight Initialization):
- Mô hình của chúng ta: Khởi tạo các trọng số của lớp tuyến tính theo phân phối chuẩn với độ lệch chuẩn là ___MATH_BLOCK_0___, và chia độ lệch chuẩn theo độ sâu cho các lớp chiếu kết nối tắt (như lớp chiếu đầu ra ___MATH_BLOCK_1___ của Attention và lớp tuyến tính thứ hai ___MATH_BLOCK_2___ của Feed-Forward):

$$\sigma = \frac{0.02}{\sqrt{2 \times \text{num_layers}}}$$

- Bài báo gốc: Khởi tạo các tham số theo phân phối chuẩn hoặc Xavier cơ bản mà không có hệ số giảm theo độ sâu ___MATH_BLOCK_0___ (đây là kỹ thuật thường thấy trong GPT-2 để khắc phục sự tích tụ phương sai ở các khối Pre-LN).

2.4. Ràng buộc chia sẻ trọng số nhúng (Weight Tying)

- Mô hình của chúng ta: Thiết lập cơ chế chia sẻ động (ở Transformer):
- Nếu kích thước từ vựng nguồn và đích bằng nhau (___CODE_BLOCK_0___), mô hình sẽ chia sẻ chung một lớp Embedding cho cả Encoder, Decoder và liên kết trực tiếp với Generator (đầu ra pre-softmax), giống hệt bài báo.

- Nếu khác nhau (__CODE_BLOCK_0__), mô hình sẽ sử dụng hai bộ Embedding riêng biệt cho nguồn và đích, chỉ chia sẻ trọng số giữa Decoder Embedding và Generator.
- Bài báo gốc: Chia sẻ hoàn toàn một ma trận trọng số nhưng cho cả Encoder Input, Decoder Input và Generator.

2.5. Cơ chế đệm động (Dynamic Padding)

- Mô hình của chúng ta: Áp dụng Dynamic Padding (đệm động). Thay vì đệm tất cả các batch về một độ dài tối đa cố định (như 80 hay 100 từ), mô hình đệm các câu trong một batch dựa trên chiều dài của câu dài nhất chỉ trong batch đó (giúp tiết kiệm đáng kể tài nguyên tính toán và bộ nhớ GPU).
- Bài báo gốc: Không đề cập hoặc đi sâu vào kỹ thuật tối ưu hóa đệm động ở cấp độ batch trong nội dung bài báo lý thuyết.

3. Khởi tạo và Cấu hình Môi trường Huấn luyện

3.1. Mã nguồn Thiết lập Nền tảng

Dưới đây là phần mã khởi tạo cấu hình môi trường và tài nguyên phần cứng cho dự án:

```
# Cấu hình môi trường PyTorch giải phóng bộ nhớ phân mảnh
import os
os.environ["PYTORCH_ALLOC_CONF"] = "expandable_segments:True"

# Cài đặt thư viện datasets của Hugging Face phục vụ tải dữ liệu
!pip install -q datasets matplotlib seaborn numpy torch tokenizers nltk

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
import math
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import re
from datasets import load_dataset

# Thiết lập seed để đảm bảo kết quả huấn luyện có thể tái lập
torch.manual_seed(42)
np.random.seed(42)

# Cấu hình thiết bị (Ưu tiên GPU CUDA nếu có, ngược lại dùng CPU)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Notebook đang chạy trên thiết bị chính: {device}")

# Kiểm tra số lượng GPU khả dụng để chuẩn bị cấu hình huấn luyện song song
num_gpus = torch.cuda.device_count()
print(f"Số lượng GPU khả dụng: {num_gpus}")
```

3.2. Ý nghĩa và Đóng góp của Từng Thành phần

1. Cấu hình chống phân mảnh bộ nhớ VRAM (__CODE_BLOCK_0__):

PyTorch quản lý bộ nhớ GPU (VRAM) bằng cách tạo sẵn các khối bộ nhớ lớn (caching) để tăng tốc độ. Tuy nhiên, khi độ dài các câu trong các batch huấn luyện thay đổi liên tục, bộ nhớ dễ bị phân mảnh (fragmentation) — tức là tổng dung lượng trống thì đủ, nhưng các khối trống lại nằm rải rác nhỏ lẻ, khiến PyTorch không thể cấp phát một khối bộ nhớ liền mạch mới. Điều này dẫn đến lỗi tràn bộ nhớ: __CODE_BLOCK_0__.

Cấu hình __CODE_BLOCK_0__ (hỗ trợ từ PyTorch 2.1+) cho phép PyTorch tự động mở rộng các khối bộ nhớ hiện có thay vì tạo mới, và giải phóng bộ nhớ phân mảnh linh hoạt hơn. Điều này cực kỳ quan trọng để chống sập mô hình (OOM) khi huấn luyện với batch size lớn trên các GPU có bộ nhớ giới hạn như Kaggle T4 (15GB).

2. Cài đặt các thư viện hỗ trợ và nạp dữ liệu:

- `__CODE_BLOCK_0__`: Thư viện của Hugging Face giúp tải nhanh bộ dữ liệu dịch máy IWSLT 2015 Việt - Anh và quản lý dữ liệu hiệu quả dưới dạng Dataset Dict.
- `__CODE_BLOCK_0__`: Thư viện tối ưu hóa bằng Rust giúp xây dựng và huấn luyện BPE Tokenizer cực nhanh.
- `__CODE_BLOCK_0__`: Thư viện xử lý ngôn ngữ tự nhiên để tính toán điểm BLEU Score đánh giá chất lượng dịch thuật.
- `__CODE_BLOCK_0__` & `__CODE_BLOCK_1__`: Dùng để vẽ các biểu đồ trực quan hóa (loss curves, learning rate, attention maps).

3. Thiết lập Seed ngẫu nhiên (`__CODE_BLOCK_0__`):

Khi khởi tạo mô hình mạng nơ-ron, các trọng số ban đầu được tạo ngẫu nhiên. Trong quá trình huấn luyện, việc xáo trộn dữ liệu (shuffle) hay cơ chế loại bỏ nơ-ron (dropout) cũng mang tính ngẫu nhiên. Việc gán giá trị cố định `__CODE_BLOCK_0__` giúp cố định tất cả các trình tạo số ngẫu nhiên này. Điều này đảm bảo tính tái lập (reproducibility): khi chạy lại mã nguồn nhiều lần hoặc chia sẻ cho người khác, mô hình sẽ luôn khởi tạo giống nhau và cho ra kết quả huấn luyện giống hệt nhau.

4. Phát hiện phần cứng tăng tốc (GPU / Multi-GPU):

Đoạn mã tự động kiểm tra xem máy tính có GPU hỗ trợ CUDA hay không. Nếu có, nó sẽ thiết lập thiết bị chạy chính là GPU (`__CODE_BLOCK_0__`), ngược lại sẽ dùng `__CODE_BLOCK_1__`. Transformer là một mô hình rất nặng. Chạy trên GPU giúp tính toán song song các ma trận Self-Attention nhanh hơn từ 10 đến 100 lần so với CPU. Việc đếm số lượng GPU (`__CODE_BLOCK_2__`) làm tiền đề để ở các bước sau, chúng ta tự động bọc mô hình bằng lớp `__CODE_BLOCK_3__` nhằm chia đôi khối lượng tính toán trên mỗi batch.

PHẦN II: CHI TIẾT CÁC THÀNH PHẦN CỐT LÕI

4. Cơ chế Mã hóa Vị trí (Positional Encoding) - Lý thuyết & Cài đặt

4.1. Sơ đồ Luồng Cộng Positional Encoding và Công thức Toán học

Dựa trên sơ đồ luồng cộng Positional Encoding trong dự án của chúng ta (sử dụng câu ví dụ "chúng ta yêu học máy" và __MATH_BLOCK_0__), chúng ta sẽ thực hiện từng bước tính toán chi tiết bằng số liệu cụ thể.

4.1.1. Tham số đầu vào từ sơ đồ:

- Câu ví dụ: "chúng ta yêu học máy"
- Vị trí 0 (__MATH_BLOCK_0__): __CODE_BLOCK_0__
- Vị trí 1 (__MATH_BLOCK_0__): __CODE_BLOCK_0__
- Vị trí 2 (__MATH_BLOCK_0__): __CODE_BLOCK_0__
- Vị trí 3 (__MATH_BLOCK_0__): __CODE_BLOCK_0__
- Số chiều mô hình: __MATH_BLOCK_0__ (chỉ số chiều đặc trưng __MATH_BLOCK_1__ chạy từ __MATH_BLOCK_2__).
- Chỉ số cặp chiều: __MATH_BLOCK_0__ chạy từ __MATH_BLOCK_1__ (với __MATH_BLOCK_2__ cho chiều chẵn và __MATH_BLOCK_3__ cho chiều lẻ).

4.1.2. Công thức toán học tổng quát:

Với mỗi vị trí __MATH_BLOCK_0__ và mỗi chiều __MATH_BLOCK_1__:

- Nếu __MATH_BLOCK_0__ là chiều chẵn (__MATH_BLOCK_1__):

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right)$$

- Nếu __MATH_BLOCK_0__ là chiều lẻ (__MATH_BLOCK_1__):

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right)$$

4.1.3. Ví dụ tính toán thực tế cho từng từ

Trường hợp 1: Từ thứ nhất __CODE_BLOCK_0__ (__MATH_BLOCK_0__)

Vì __MATH_BLOCK_0__ nên từ số của góc luôn bằng __MATH_BLOCK_1__.

- Với các chiều chẵn (__MATH_BLOCK_0__):

$$PE_{(0, 2i)} = \sin\left(\frac{0}{10000^{\frac{2i}{256}}}\right) = \sin(0) = 0$$

- Với các chiều lẻ (__MATH_BLOCK_0__):

$$PE_{(0, 2i+1)} = \cos\left(\frac{0}{10000^{\frac{2i}{256}}}\right) = \cos(0) = 1$$

__MATH_BLOCK_0__ Kết quả Vector PE của từ __CODE_BLOCK_0__ (__MATH_BLOCK_1__) luôn cố định là:

$$PE_{(pos=0)} = [0, 1, 0, 1, 0, 1, \dots, 0, 1]$$

Trường hợp 2: Từ thứ hai `__CODE_BLOCK_0__` (`__MATH_BLOCK_0__`)

Chúng ta sẽ tính giá trị PE tại các vị trí chiều khác nhau trong vector `__MATH_BLOCK_0__` chiều:

- Tại hai chiều đầu tiên (`__MATH_BLOCK_0__` và `__MATH_BLOCK_1__`):
- Tỷ lệ tần số: `__MATH_BLOCK_0__`
- Chiều chẵn (`__MATH_BLOCK_0__`):

$$PE_{(1, 0)} = \sin\left(\frac{1}{1}\right) = \sin(1 \text{ rad}) \approx 0.8415$$

- Chiều lẻ (`__MATH_BLOCK_0__`):

$$PE_{(1, 1)} = \cos\left(\frac{1}{1}\right) = \cos(1 \text{ rad}) \approx 0.5403$$

- Tại hai chiều tiếp theo (`__MATH_BLOCK_0__` và `__MATH_BLOCK_1__`):
- Tỷ lệ tần số: `__MATH_BLOCK_0__`
- Chiều chẵn (`__MATH_BLOCK_0__`):

$$PE_{(1, 2)} = \sin\left(\frac{1}{1.0746}\right) \approx \sin(0.9306 \text{ rad}) \approx 0.8018$$

- Chiều lẻ (`__MATH_BLOCK_0__`):

$$PE_{(1, 3)} = \cos\left(\frac{1}{1.0746}\right) \approx \cos(0.9306 \text{ rad}) \approx 0.5976$$

- Tại hai chiều cuối cùng của vector (`__MATH_BLOCK_0__` và `__MATH_BLOCK_1__`):
- Tỷ lệ tần số: `__MATH_BLOCK_0__`
- Chiều chẵn (`__MATH_BLOCK_0__`):

$$PE_{(1, 254)} = \sin\left(\frac{1}{9194.79}\right) \approx \sin(0.0001087) \approx 0.0001087$$

- Chiều lẻ (`__MATH_BLOCK_0__`):

$$PE_{(1, 255)} = \cos\left(\frac{1}{9194.79}\right) \approx \cos(0.0001087) \approx 1.0000$$

`__MATH_BLOCK_0__` Kết quả Vector PE của từ `__CODE_BLOCK_0__` (`__MATH_BLOCK_1__`) sẽ là:

$$PE_{(pos = 1)} = [0.8415, 0.5403, 0.8018, 0.5976, \dots, 0.0001, 1.0000]$$

4.1.4. Ý nghĩa của phép cộng luồng trong sơ đồ:

Cơ chế cộng Luồng Positional Encoding	Sơ đồ luồng cộng
Sau khi tính toán xong vector <code>__MATH_BLOCK_0__</code> (ví dụ kích thước <code>__MATH_BLOCK_1__</code> cho từ "yêu" tại <code>__MATH_BLOCK_2__</code>), mô hình thực hiện phép cộng trực tiếp (element-wise) vào vector những từ (Word Embedding <code>__MATH_BLOCK_3__</code>): <code>__MATH_BLOCK_4__</code> Lưu ý quan trọng: Ta nhân tỷ lệ <code>__MATH_BLOCK_5__</code> vào Word Embedding trước khi cộng để giữ cho thông tin vị trí không lấn át thông tin ngữ nghĩa của từ. Đồng thời, điều này giúp cân bằng phương sai giữa phần biểu diễn ngữ nghĩa và biểu diễn vị trí.	* Sơ đồ luồng cộng Positional Encoding

4.2. Tại sao lại sử dụng công thức cơ số `__MATH_BLOCK_0__`?

Con số `__MATH_BLOCK_0__` không phải là một con số ngẫu nhiên mà là một thiết kế toán học cực kỳ xuất sắc của các tác giả bài báo Transformer.

Nó đóng vai trò xác định tần số sóng (frequency) cho từng chiều trong vector `__MATH_BLOCK_0__` chiều. Để giải thích một cách dễ hiểu và thuyết phục nhất cho người đánh giá, chúng ta có thể trình bày theo 3 khía cạnh sau:

4.2.1. Ý nghĩa Vật lý: Phân rã tần số từ nhanh đến chậm

Nếu ta gọi tần số của chiều thứ `__MATH_BLOCK_0__` là `__MATH_BLOCK_1__`, thì khi chiều `__MATH_BLOCK_2__` chạy từ `__MATH_BLOCK_3__`:

- Ở những chiều đầu tiên (`__MATH_BLOCK_0__`): Tần số `__MATH_BLOCK_1__`. Sóng sin/cos biến đổi cực kỳ nhanh (chu kỳ cực ngắn chỉ khoảng `__MATH_BLOCK_2__` từ).

`__MATH_BLOCK_0__` Vai trò: Giúp mô hình nhận biết các mối quan hệ rất gần (ví dụ: từ đứng ngay trước hoặc ngay sau từ hiện tại).

- Ở những chiều cuối cùng (`__MATH_BLOCK_0__`): Tần số `__MATH_BLOCK_1__`. Sóng biến đổi cực kỳ chậm (chu kỳ cực dài lên tới hơn `__MATH_BLOCK_2__` từ).

`__MATH_BLOCK_0__` Vai trò: Giúp mô hình nhận biết cấu trúc toàn cục và khoảng cách xa (ví dụ: từ này nằm ở đầu câu hay cuối câu).

Hình ảnh ẩn dụ dễ hiểu: Nó giống như mặt đồng hồ kim. Kim giây (`__MATH_BLOCK_0__` nhỏ) quay rất nhanh để chỉ thời gian chi tiết từng giây, kim phút quay vừa, còn kim giờ (`__MATH_BLOCK_1__` lớn) quay rất chậm để chỉ tổng thể thời gian trong ngày. Cả ba kim kết hợp lại sẽ cho ta biết chính xác thời gian là bao nhiêu.

4.2.2. Tại sao cơ số lại là `__MATH_BLOCK_0__`?

- Chu kỳ của hàm hình sin là `__MATH_BLOCK_0__`. Để các vector vị trí không bị trùng lặp (ví dụ vị trí thứ `__MATH_BLOCK_1__` có vector giống hệt vị trí thứ `__MATH_BLOCK_2__`), chu kỳ lớn nhất của sóng phải lớn hơn độ dài của bất kỳ câu nào trong thực tế.
- Với cơ số `__MATH_BLOCK_0__`, bước sóng dài nhất sẽ là:

$$\lambda_{max} = 2\pi \times 10000 \approx 62,831 \text{ tokens}$$

- Vì các câu dịch thông thường chỉ dài dưới 500 từ, việc chọn cơ số `__MATH_BLOCK_0__` đảm bảo không bao giờ xảy ra hiện tượng trùng lặp pha (aliasing/ambiguity) ở các chiều có tần số chậm, giúp mỗi vị trí trong câu luôn có một "chữ ký" độc nhất vô nhị.

4.2.3. Tại sao số mũ lại là tỷ lệ tuyến tính `__MATH_BLOCK_0__`?

Tác giả muốn cơ chế Attention có thể dễ dàng học được khoảng cách tương đối giữa các từ (từ `__MATH_BLOCK_0__` cách từ `__MATH_BLOCK_1__` bao nhiêu từ), chứ không chỉ là vị trí tuyệt đối.

Nhờ công thức lượng giác:

$$\sin(a + b) = \sin(a)\cos(b) + \cos(a)\sin(b)$$

$$\cos(a + b) = \cos(a)\cos(b) - \sin(a)\sin(b)$$

Với một khoảng cách dịch chuyển `__MATH_BLOCK_0__` cố định (ví dụ: từ cách nhau `__MATH_BLOCK_1__` vị trí), vector vị trí ở `__MATH_BLOCK_2__` hoàn toàn có thể được biểu diễn dưới dạng phép biến đổi tuyến tính (nhân ma trận xoay) của vector vị trí tại `__MATH_BLOCK_3__`:

$$PE_{(pos+k)} = M_k \times PE_{(pos)}$$

Trong đó ma trận chuyển đổi `__MATH_BLOCK_0__` chỉ phụ thuộc vào khoảng cách `__MATH_BLOCK_1__` mà không phụ thuộc vào vị trí tuyệt đối `__MATH_BLOCK_2__`. Số mũ tỷ lệ tuyến tính `__MATH_BLOCK_3__` chính là điều kiện toán học bắt buộc để phép biến đổi ma trận xoay `__MATH_BLOCK_4__` này tồn tại và hoạt động trơn tru trên mọi chiều ẩn.

4.3. Tại sao lại sử dụng sóng Sin và Cosine?

Việc sử dụng hàm sóng hình sin (Sine) và hình cosin (Cosine) cho Positional Encoding giải quyết được 3 vấn đề toán học và kỹ thuật rất lớn mà các phương pháp khác không làm được:

4.3.1. Học khoảng cách tương đối bằng Phép quay Tuyến tính

Trong cơ chế Self-Attention, mô hình cần biết các từ cách nhau bao xa để liên kết ngữ cảnh (ví dụ: từ cách nhau 1 từ, 2 từ...).

Bằng cách xếp xen kẽ Sine ở chiều chẵn và Cosine ở chiều lẻ cho cùng một tần số `__MATH_BLOCK_0__`, ta tạo ra các cặp tọa độ lượng giác `__MATH_BLOCK_1__`.

Khi vị trí dịch chuyển từ `__MATH_BLOCK_0__`, vector lượng giác này sẽ xoay một góc là `__MATH_BLOCK_1__`.

Nhờ công thức cộng lượng giác:

$$\sin(pos+k) = \sin(pos)\cos(k) + \cos(pos)\sin(k)$$

$$\cos(pos+k) = \cos(pos)\cos(k) - \sin(pos)\sin(k)$$

Mô hình chỉ cần dùng một ma trận biến đổi tuyến tính (ma trận xoay) để ánh xạ từ vị trí `__MATH_BLOCK_0__` sang vị trí `__MATH_BLOCK_1__`. Lớp tuyến tính (Linear Layer) trong mạng nơ-ron học phép toán này cực kỳ dễ dàng.

4.3.2. Tự động tính khoảng cách qua tích vô hướng

Cơ chế Attention tính toán độ liên quan giữa các từ bằng tích vô hướng (Dot Product) của các vector.

Khi ta nhân vô hướng hai vector vị trí `__MATH_BLOCK_0__` và `__MATH_BLOCK_1__`, nhờ công thức:

$$\cos(pos_1)\cos(pos_2) + \sin(pos_1)\sin(pos_2) = \cos(pos_1 - pos_2)$$

Kết quả tích vô hướng giữa hai mã hóa vị trí chỉ phụ thuộc vào khoảng cách tương đối giữa chúng (`__MATH_BLOCK_0__`), chứ không phụ thuộc vào vị trí tuyệt đối nằm ở đâu trong câu. Điều này giúp cơ chế Attention tự động biết được khoảng cách vật lý giữa hai từ một cách tự nhiên.

4.3.3. Khả năng ngoại suy độ dài câu không giới hạn

- Nếu ta dùng phương pháp học vị trí (Learned Positional Embeddings) như trong BERT hay GPT: Ta phải định nghĩa trước một độ dài tối đa (ví dụ: tối đa 512 từ). Khi chạy thực tế gặp câu dài 513 từ, mô hình sẽ bị lỗi lập tức vì không có vector vị trí thứ 513 để nạp vào.
- Với hàm Sin/Cos: Đây là các hàm toán học liên tục, xác định trên toàn bộ trục số thực từ `__MATH_BLOCK_0__`. Do đó, mô hình có thể tính toán vector vị trí cho câu dài bao nhiêu tùy ý (1000 từ, 2000 từ...) mà không bao giờ bị lỗi crash hệ thống ở bước suy luận (inference).

4.3.4. Giới hạn biên giá trị và tính liên tục

- Nếu ta đánh số vị trí dạng số nguyên tuyệt đối (`__MATH_BLOCK_0__`): Khi câu quá dài, giá trị vị trí sẽ cực lớn, làm mất ổn định quá trình tối ưu hóa gradient.

- Nếu ta chuẩn hóa vị trí về đoạn `__MATH_BLOCK_0__` (bằng cách lấy `__MATH_BLOCK_1__`): Khi đó, vị trí `__MATH_BLOCK_2__` của câu dài 10 từ sẽ là từ thứ 5, nhưng của câu dài 100 từ sẽ là từ thứ 50. Ý nghĩa vị trí bị thay đổi tùy thuộc vào chiều dài câu, gây bối rối cho mô hình.
- Hàm Sin/Cos luôn giới hạn giá trị đầu ra trong khoảng `__MATH_BLOCK_0__` (rất thân thiện với gradient descent) và giữ nguyên khoảng cách tuyệt đối giữa các từ bất kể câu dài hay ngắn.

4.4. Giải thích Mã nguồn lớp `PositionalEncoding` trong `PyTorch`

4.4.1. Ý nghĩa của các con số trong code

- Con số `__CODE_BLOCK_0__` (`__CODE_BLOCK_1__`):
 - Ý nghĩa: Đây là độ dài tối đa của câu mà lớp này chuẩn bị sẵn ma trận vị trí.
 - Tại sao lại là 5000? Hầu hết các câu trong thực tế (như bộ IWSLT15 Việt - Anh) chỉ dài tối đa dưới 100-200 từ. Việc chọn `__MATH_BLOCK_0__` là một con số "dư dả" để đảm bảo mô hình không bao giờ bị thiếu ma trận vị trí khi xử lý các đoạn văn dài trong tương lai.
- Con số `__CODE_BLOCK_0__` (`__CODE_BLOCK_1__`):
 - Ý nghĩa: Hệ số cơ số (Base) để tính toán tần số sóng lượng giác. Như đã giải thích ở câu hỏi trước, con số `__MATH_BLOCK_0__` giúp tạo ra bước sóng cực kỳ dài cho các chiều ẩn cuối cùng, ngăn chặn hiện tượng trùng lặp vị trí đối với các câu dài.
- Bước nhảy `__CODE_BLOCK_0__` (`__CODE_BLOCK_1__` và lát cắt `__CODE_BLOCK_2__`, `__CODE_BLOCK_3__`):
 - Ý nghĩa: Chia đôi chiều `__MATH_BLOCK_0__` thành chẵn và lẻ.
 - `__CODE_BLOCK_0__` nghĩa là lấy các cột chẵn (`__MATH_BLOCK_0__`) để điền hàm Sin.
 - `__CODE_BLOCK_0__` nghĩa là lấy các cột lẻ (`__MATH_BLOCK_0__`) để điền hàm Cos.

4.4.2. Giải thích chi tiết từng dòng code

1. Khởi tạo ma trận rỗng

```
pe = torch.zeros(max_len, d_model)
```

- Tạo ma trận chứa toàn số 0 kích thước `__CODE_BLOCK_0__` làm khung để chuẩn bị điền các giá trị mã hóa vị trí vào.

2. Tạo vector vị trí

```
position = torch.arange(0, max_len, dtype=torch.float).unsqueeze(1)
```

- `__CODE_BLOCK_0__` tạo ra dãy số nguyên từ `__MATH_BLOCK_0__` đại diện cho vị trí các từ trong câu.
- `__CODE_BLOCK_0__` biến vector hàng thành vector cột có kích thước `__CODE_BLOCK_1__`.

3. Tính toán `div_term` (mẫu số của góc)

```
div_term = torch.exp(torch.arange(0, d_model, 2).float() * (-math.log(10000.0) / d_model))
```

- `__CODE_BLOCK_0__` tạo ra dãy số chẵn đại diện cho các chỉ số `__MATH_BLOCK_0__`.
- `__CODE_BLOCK_0__` chính là nhân với cụm `__MATH_BLOCK_0__`.
- `__CODE_BLOCK_0__` lấy hàm mũ `__MATH_BLOCK_0__`.
- Kết quả: Tính ra chính xác giá trị mẫu số cho mọi chiều ẩn cực kỳ nhanh và ổn định về mặt số học.

4. Điền giá trị Sin và Cos

```
pe[:, 0::2] = torch.sin(position * div_term)
pe[:, 1::2] = torch.cos(position * div_term)
```

- Nhân vector cột `__CODE_BLOCK_0__` `__CODE_BLOCK_1__` với vector hàng `__CODE_BLOCK_2__` `__CODE_BLOCK_3__` để tạo ma trận góc.
- Áp dụng hàm `__CODE_BLOCK_0__` cho các cột chẵn (`__CODE_BLOCK_1__`) và `__CODE_BLOCK_2__` cho các cột lẻ (`__CODE_BLOCK_3__`).

5. Thêm chiều Batch Size và Đăng ký Buffer

```
pe = pe.unsqueeze(0)
self.register_buffer('pe', pe)
```

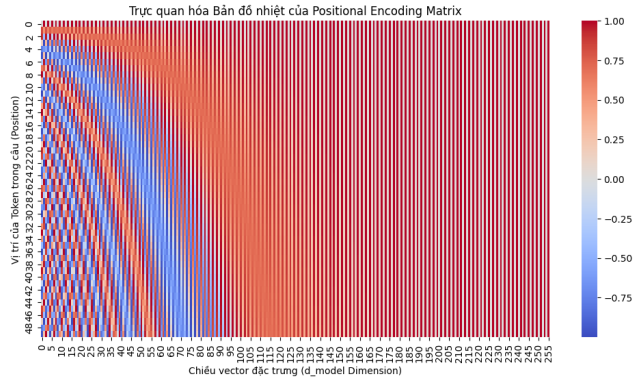
- `__CODE_BLOCK_0__` biến ma trận `__CODE_BLOCK_1__` thành `__CODE_BLOCK_2__`. Chiều đầu tiên đại diện cho Batch Size. Nhờ cơ chế tự phát sóng (broadcasting) của PyTorch, khi cộng với tensor đầu vào `__CODE_BLOCK_3__` có kích thước `__CODE_BLOCK_4__`, chiều `__CODE_BLOCK_5__` này tự động lặp lại cho khớp với `__CODE_BLOCK_6__`.
- `__CODE_BLOCK_0__` đăng ký ma trận này thành một Buffer tĩnh của mô hình. Tức là:
 1. Nó không được tính gradient và không được tối ưu hóa (vì vị trí là cố định).
 2. Nó sẽ tự động lưu lại khi chúng ta lưu mô hình (`__CODE_BLOCK_0__`).
 3. Nó sẽ tự động được chuyển lên GPU hoặc CPU đồng bộ khi chúng ta gọi `__CODE_BLOCK_0__`.

6. Phép cộng ở hàm Forward

```
def forward(self, x):
    x = x + self.pe[:, :x.size(1)]
    return x
```

- `__CODE_BLOCK_0__` lấy độ dài thực tế của câu trong batch hiện tại (ví dụ batch này câu dài nhất là 15 từ thì `__CODE_BLOCK_1__`).
- `__CODE_BLOCK_0__` cắt ma trận PE đã chuẩn bị sẵn từ kích thước `__MATH_BLOCK_0__` xuống đúng độ dài `__MATH_BLOCK_1__` để thực hiện phép cộng ma trận tương ứng.

4.5. Giải thích Biểu đồ Nhiệt Positional Encoding (Positional Encoding Heatmap)

Phân tích Bản đồ nhiệt Positional Encoding	Bản đồ nhiệt lượng tử (Heatmap)
Biểu đồ trực quan hóa Ma trận mã hóa vị trí (Positional Encoding Matrix) kích thước <code>__MATH_BLOCK_0__</code> (<code>__MATH_BLOCK_1__</code> từ đầu tiên và vector những vị trí <code>__MATH_BLOCK_2__</code> chiều). 1. Trục tọa độ và màu sắc: • Trục tung (Y-axis): Vị trí của từ trong câu (<code>__MATH_BLOCK_3__</code> từ <code>__MATH_BLOCK_4__</code>). Hàng ngang là vector vị trí của từ cụ thể. • Trục hoành (X-axis): Chiều của vector đặc trưng (<code>__MATH_BLOCK_5__</code> từ <code>__MATH_BLOCK_6__</code>). • Thang màu (Colorbar): Hàm Sin/Cos từ <code>__MATH_BLOCK_7__</code> (Xanh) đến <code>__MATH_BLOCK_8__</code> (Đỏ đậm). Màu cam/trắng biểu thị gần <code>__MATH_BLOCK_9__</code>. 2. Sự phân bố tần số sóng: • Vùng bên trái (chiều <code>__MATH_BLOCK_10__</code>): Các sọc dọc nhuộm đan xen dày đặc do tần số lớn (bước sóng ngắn). Giúp mô hình nhận biết khoảng cách chỉ tiết cự ly gần. • Vùng bên phải (chiều <code>__MATH_BLOCK_11__</code>): Dải màu đồng nhất, biến đổi rất ít do tần số cực nhỏ (bước sóng rất dài). Giúp nhận biết vị trí vĩ mô (phần đầu/phần cuối câu). 3. Quy luật độc bản: • Cắt ngang qua bất kỳ hàng <code>__MATH_BLOCK_12__</code> nào ta sẽ thu được tổ hợp màu sắc "độc nhất vô nhị". Tổ hợp sóng sin/cos tần số khác nhau tạo ra một "chữ ký lượng tử" (signature) duy nhất cho mỗi vị trí, giúp mô hình không bao giờ nhầm lẫn thứ tự các từ.	Trực quan hóa Bản đồ nhiệt của Positional Encoding Matrix  * Bản đồ nhiệt Positional Encoding Heatmap

Tích vô hướng đo độ tương đồng

Mục tiêu của Attention là đo "từ này liên quan đến từ kia bao nhiêu?". Công cụ toán học cơ bản nhất để đo độ tương đồng giữa hai vector là tích vô hướng (Dot Product):

$$\text{similarity}(\vec{a}, \vec{b}) = \vec{a} \cdot \vec{b} = a_1b_1 + a_2b_2 + \dots + a_nb_n$$

- Nếu hai vector chỉ cùng hướng (cùng ngữ nghĩa) → tích vô hướng lớn (điểm cao).
- Nếu hai vector vuông góc (không liên quan) → tích vô hướng bằng 0.
- Nếu hai vector ngược hướng (nghĩa đối lập) → tích vô hướng âm.

4.6. Tại sao phải chuyển vị `__MATH_BLOCK_0__`?

Giả sử ta có câu gồm 3 từ, mỗi từ được biểu diễn bằng vector `__MATH_BLOCK_0__` chiều:

$$Q = \begin{bmatrix} q_1 \\ q_2 \\ q_3 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 1 & 0 & 0 \end{bmatrix} \quad (\text{kích thước } 3 \times 4)$$

$$K = \begin{bmatrix} k_1 \\ k_2 \\ k_3 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 \end{bmatrix} \quad (\text{kích thước } 3 \times 4)$$

Nếu ta nhân trực tiếp `__MATH_BLOCK_0__` (kích thước `__MATH_BLOCK_1__` nhân `__MATH_BLOCK_2__`) → Không thể nhân được! Vì số cột của `__MATH_BLOCK_3__` (`__MATH_BLOCK_4__`) khác số hàng của `__MATH_BLOCK_5__` (`__MATH_BLOCK_6__`).

Khi chuyển vị `__MATH_BLOCK_0__`:

$$K^T = \begin{bmatrix} 1 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 1 \end{bmatrix} \quad (\text{kích thước } 4 \times 3)$$

Lúc này `__MATH_BLOCK_0__` có kích thước `__MATH_BLOCK_1__` → Hoàn hảo! Ma trận kết quả `__MATH_BLOCK_2__` chính là ma trận điểm tương đồng giữa mọi cặp từ trong câu:

4.7. Ví dụ tính toán chi tiết bằng số của cơ chế Self-Attention

Để hiểu rõ hơn, chúng ta hãy đi qua một ví dụ cụ thể. Giả sử ta có một câu gồm 3 từ: `__CODE_BLOCK_0__`, `__CODE_BLOCK_1__`, `__CODE_BLOCK_2__`. Mỗi từ được đại diện bằng một vector `__MATH_BLOCK_0__` chiều.

Giả sử ta đã tính toán được ma trận `__MATH_BLOCK_0__`, `__MATH_BLOCK_1__`, và `__MATH_BLOCK_2__` (kích thước `__MATH_BLOCK_3__`):

$$Q = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 1 & 0 & 0 \end{bmatrix} \quad (\text{kích thước } 3 \times 4)$$

$$K = \begin{bmatrix} 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 \end{bmatrix} \quad (\text{kích thước } 3 \times 4)$$

$$V = \begin{bmatrix} v_{11} & v_{12} & v_{13} & v_{14} \\ v_{21} & v_{22} & v_{23} & v_{24} \\ v_{31} & v_{32} & v_{33} & v_{34} \end{bmatrix} \quad (\text{kích thước } 3 \times 4)$$

Bước 1: Tính tích vô hướng __MATH_BLOCK_0__

Chuyển vị ma trận __MATH_BLOCK_0__ thành __MATH_BLOCK_1__ có kích thước __MATH_BLOCK_2__:

$$K^T = \begin{bmatrix} 1 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 1 \end{bmatrix} \quad (\text{kích thước } 4 \times 3)$$

Thực hiện phép nhân ma trận __MATH_BLOCK_0__:

$$Scores = Q \times K^T = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 1 & 0 & 0 \end{bmatrix} \times \begin{bmatrix} 1 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 2 & 0 & 1 \end{bmatrix}$$

Bước 2: Chia cho __MATH_BLOCK_0__ (với __MATH_BLOCK_1__)

$$\frac{Scores}{2} = \frac{1}{2} \times \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 2 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0.5 & 0.5 & 0.5 \\ 0.5 & 0.5 & 0.5 \\ 1.0 & 0.0 & 0.5 \end{bmatrix}$$

Bước 3: Áp dụng hàm Softmax để tính trọng số Attention (Attention Weights)

Hàm Softmax được áp dụng theo từng hàng của ma trận điểm số để tạo ra phân phối xác suất (tổng mỗi hàng bằng __MATH_BLOCK_0__).

Giả sử sau khi tính toán Softmax, ta thu được ma trận trọng số chú ý:

$$AttentionWeights = \begin{bmatrix} 0.35 & 0.42 & 0.23 \\ 0.30 & 0.25 & 0.45 \\ 0.37 & 0.45 & 0.18 \end{bmatrix}$$

Bước 4: Nhân ma trận trọng số với ma trận giá trị __MATH_BLOCK_0__ để thu được vector ngữ cảnh (Context Vector)

$$\text{Context} = \text{AttentionWeights} \begin{bmatrix} 0.350 & 0.420 & 0.23 \\ 0.30 & 0.25 & 0.45 \\ 0.370 & 0.450 & 0.18 \end{bmatrix} \times \begin{bmatrix} V_{11} & V_{12} & V_{13} & V_{14} \\ V_{21} & V_{22} & V_{23} & V_{24} \\ V_{31} & V_{32} & V_{33} & V_{34} \end{bmatrix} = \begin{bmatrix} 0.35\vec{v}_1 + 0.42\vec{v}_2 + 0.23\vec{v}_3 \\ 0.30\vec{v}_1 + 0.25\vec{v}_2 + 0.45\vec{v}_3 \\ 0.37\vec{v}_1 + 0.45\vec{v}_2 + 0.18\vec{v}_3 \end{bmatrix}$$

Ý nghĩa: Vector mới của từ `__CODE_BLOCK_0__` bây giờ là trung bình có trọng số của nội dung (Value) của cả 3 từ, trong đó thông tin từ `__CODE_BLOCK_1__` chiếm 42% (quan trọng nhất). Đây chính là cách mô hình "pha trộn" ngữ cảnh vào vector từ.

Bước 3: Ghép các Heads lại và Chiều đầu ra

```
context = context.transpose(1, 2).contiguous().view(batch_size, -1, self.d_model)
output = self.W_o(context)
```

```
Head 1 Context: [1, 3, 4]      Head 2 Context: [1, 3, 4]
  ↓ transpose + view (ghép 2 heads lại thành 8 chiều)
Concatenated:   [1, 3, 8]      ← Mỗi từ giờ có vector 8 chiều (= d_model)
  ↓ W_o (nhân ma trận trọng số 8×8)
Output:         [1, 3, 8]      ← Vector đầu ra cuối cùng
```

Ý nghĩa: Head 1 có thể đã học cách chú ý đến quan hệ cú pháp (chủ-vị), Head 2 học cách chú ý đến quan hệ vị trí gần. Lớp `__MATH_BLOCK_0__` kết hợp thông tin từ cả hai "góc nhìn" này thành một biểu diễn thống nhất cuối cùng.

Tóm tắt dòng chảy dữ liệu toàn bộ:

```
Đầu vào x [1, 3, 8]
  ↓ Chiều tuyến tính W_q, W_k, W_v
Q, K, V [1, 3, 8]
  ↓ Chia thành nhiều heads
Q, K, V [1, 2, 3, 4]      ← 2 heads, mỗi head 4 chiều
  ↓ Q × K^T / √d_k
Scores [1, 2, 3, 3]      ← Ma trận tương đồng 3×3 cho mỗi head
  ↓ Mask + Softmax + Dropout
Weights [1, 2, 3, 3]      ← Trọng số chú ý (tổng mỗi hàng = 1)
  ↓ Weights × V
Context [1, 2, 3, 4]      ← Vector ngữ cảnh cho mỗi head
  ↓ Ghép heads + Chiều W_o
Output [1, 3, 8]          ← Vector đầu ra cuối cùng (cùng kích thước đầu vào)
```

5. Cơ chế Multi-Head Self-Attention - Lý thuyết & Cài đặt

5.1. Giải thích chi tiết mã nguồn MultiHeadAttention

5.1.1. Hàm khởi tạo `__CODE_BLOCK_0__`

1.1. Kế thừa lớp cha

```
class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, num_heads, dropout=0.1):
        super(MultiHeadAttention, self).__init__()
```

- `__CODE_BLOCK_0__` là lớp cơ sở của mọi module mạng nơ-ron trong PyTorch. Khi kế thừa, ta được PyTorch tự động quản lý các trọng số (parameters), lưu/tải mô hình, chuyển GPU/CPU...
- `__CODE_BLOCK_0__` gọi hàm khởi tạo của lớp cha để PyTorch đăng ký module này vào hệ thống quản lý.

- Tham số đầu vào:
- `__CODE_BLOCK_0__`: Số chiều vector nhúng của mỗi từ (ví dụ: `__MATH_BLOCK_0__`).
- `__CODE_BLOCK_0__`: Số lượng đầu chú ý song song (ví dụ: `__MATH_BLOCK_0__`).
- `__CODE_BLOCK_0__`: Tỷ lệ loại bỏ ngẫu nhiên 10% các trọng số chú ý trong quá trình huấn luyện.

1.2. Kiểm tra ràng buộc chia hết

```
assert d_model % num_heads == 0, "d_model phải chia hết cho num_heads"
```

- Tại sao? Vì ta sẽ chia đều `__MATH_BLOCK_0__` chiều cho `__MATH_BLOCK_1__` đầu. Nếu không chia hết (ví dụ `__MATH_BLOCK_2__`), ta không thể phân bổ đều số chiều cho mỗi head.
- Ví dụ hợp lệ: `__MATH_BLOCK_0__` chiều/head
- Ví dụ không hợp lệ: `__MATH_BLOCK_0__` → Chương trình sẽ dừng lại ngay.

1.3. Lưu trữ các tham số cấu hình

```
self.d_model = d_model
self.num_heads = num_heads
self.d_k = d_model // num_heads
```

- `__CODE_BLOCK_0__` là số chiều đặc trưng mà mỗi head được phân bổ.
- Ví dụ: `__MATH_BLOCK_0__`, `__MATH_BLOCK_1__` → `__MATH_BLOCK_2__`.
- Phép `__CODE_BLOCK_0__` là chia lấy phần nguyên trong Python.

1.4. Định nghĩa 4 lớp chiếu tuyến tính

```
self.W_q = nn.Linear(d_model, d_model)
self.W_k = nn.Linear(d_model, d_model)
self.W_v = nn.Linear(d_model, d_model)
self.W_o = nn.Linear(d_model, d_model)
self.dropout = nn.Dropout(dropout)
```

- `__CODE_BLOCK_0__`, `__CODE_BLOCK_1__`, `__CODE_BLOCK_2__`: 3 lớp chiếu tuyến tính dùng để biến đổi vector nhúng đầu vào thành các vector Query, Key, Value tương ứng.
- `__CODE_BLOCK_0__`: Lớp chiếu tuyến tính đầu ra dùng sau khi nối các heads lại để đưa kích thước dữ liệu về lại `__MATH_BLOCK_0__`.
- `__CODE_BLOCK_0__`: Lớp Dropout áp dụng lên ma trận trọng số chú ý để tránh hiện tượng quá khớp.

5.1.2. Hàm Forward (Truyền xuôi) và Luồng xử lý

Hàm `__CODE_BLOCK_0__` thực hiện việc tính toán Attention đa đầu:

```
def forward(self, q, k, v, mask=None):
    batch_size = q.size(0)

    Q = self.W_q(q).view(batch_size, -1, self.num_heads, self.d_k).transpose(1, 2)
    K = self.W_k(k).view(batch_size, -1, self.num_heads, self.d_k).transpose(1, 2)
    V = self.W_v(v).view(batch_size, -1, self.num_heads, self.d_k).transpose(1, 2)
```

2.1. Chiếu tuyến tính và Chia Heads

- Đầu tiên, các ma trận `__CODE_BLOCK_0__`, `__CODE_BLOCK_1__`, `__CODE_BLOCK_2__` được nhân với các trọng số tuyến tính tương ứng.

- `__CODE_BLOCK_0__` phân tách chiều `__MATH_BLOCK_0__` thành `__CODE_BLOCK_1__` đầu, mỗi đầu có số chiều là `__CODE_BLOCK_2__`.
- `__CODE_BLOCK_0__` chuyển vị chiều thứ 1 (độ dài chuỗi) và chiều thứ 2 (số đầu chú ý), đưa ma trận về dạng `__CODE_BLOCK_1__`.

2.2. Tính toán điểm số chú ý

```
scores = torch.matmul(Q, K.transpose(-2, -1)) / math.sqrt(self.d_k)
```

- `__CODE_BLOCK_0__` tính tích vô hướng giữa các vector Query và Key của tất cả các từ trong câu.
- `__CODE_BLOCK_0__` thực hiện chia cho `__MATH_BLOCK_0__` để tránh các điểm số quá lớn, làm giảm gradient khi đi qua hàm Softmax.

2.3. Áp dụng Mask

```
if mask is not None:
    scores = scores.masked_fill(mask == 0, -1e4)
```

- Nếu có mặt nạ `__CODE_BLOCK_0__`, các vị trí có giá trị bằng `__CODE_BLOCK_1__` (là các token `__CODE_BLOCK_2__` hoặc các từ ở tương lai mà Decoder không được nhìn thấy) sẽ bị gán một giá trị âm cực lớn (`__MATH_BLOCK_0__`).
- Khi đi qua hàm Softmax, các vị trí này sẽ có giá trị xác suất bằng 0.

2.4. Tính toán trọng số chú ý

```
attn_weights = self.dropout(torch.softmax(scores, dim=-1))
```

- `__CODE_BLOCK_0__` chuyển đổi các điểm số attention thành phân phối xác suất
- `__CODE_BLOCK_0__`: Ngẫu nhiên tắt 10% trọng số chú ý (chỉ khi huấn luyện).

2.5. Nhân với Value

```
context = torch.matmul(attn_weights, V)
```

Phép toán	Kích thước	Giải thích
<code>__CODE_BLOCK_0__</code>	<code>__CODE_BLOCK_0__</code>	Trọng số chú ý (mỗi hàng tổng = 1)
<code>__CODE_BLOCK_0__</code>	<code>__CODE_BLOCK_0__</code>	Nội dung (Value) của từng từ
<code>__CODE_BLOCK_0__</code>	<code>__CODE_BLOCK_0__</code>	Trung bình có trọng số của tất cả vector Value. Từ nào được chú ý nhiều (trọng số cao) sẽ đóng góp nhiều nội dung hơn vào vector kết quả

2.6. Ghép heads và Chiếu đầu ra

```
context = context.transpose(1, 2).contiguous().view(batch_size, -1, self.d_model)
output = self.W_o(context)
```

Bước	Phép toán	Kích thước	Giải thích
1	__CODE_BLOCK_0__	__CODE_BLOCK_0__	Hoán đổi lại __CODE_BLOCK_0__ và __CODE_BLOCK_1__ về đúng thứ tự ban đầu
2	__CODE_BLOCK_0__	__CODE_BLOCK_0__	Sắp xếp lại vùng nhớ liên tục sau phép transpose (bắt buộc trước khi gọi __CODE_BLOCK_0__)
3	__CODE_BLOCK_0__	__CODE_BLOCK_0__	Ghép 8 heads × 64 chiều = 512 chiều. Đây chính là phép Concatenate trong công thức __MATH_BLOCK_0__
4	__CODE_BLOCK_0__	__CODE_BLOCK_0__	Chiều tuyến tính cuối cùng (__MATH_BLOCK_0__) để kết hợp thông tin từ tất cả heads thành biểu diễn thống nhất

2.7. Trả về kết quả

```
return output, attn_weights
```

- __CODE_BLOCK_0__ __CODE_BLOCK_1__: Vector biểu diễn mới của mỗi từ sau khi đã tích hợp ngữ cảnh từ các từ khác.
- __CODE_BLOCK_0__ __CODE_BLOCK_1__: Ma trận trọng số chú ý (dùng để vẽ Attention Map trực quan hóa mô hình đang chú ý vào đâu).

Giải thích Luồng tính toán Multi-Head Attention

Quy trình biến đổi ma trận để thu được vector ngữ cảnh trên từng đầu con (Head):

- Bước 1 — Chiều & Chia Heads: Đầu vào __MATH_BLOCK_0__ kích thước __CODE_BLOCK_0__ chiều tuyến tính thành __MATH_BLOCK_1__, sau đó tách và chuyển vị thành __CODE_BLOCK_1__ để tính toán song song trên __MATH_BLOCK_2__ heads độc lập.
- Bước 2 — Tính Scaled Dot-Product: Nhân tích vô hướng __MATH_BLOCK_3__, chia tỷ lệ __MATH_BLOCK_4__, áp dụng Mask và Softmax để thu được ma trận trọng số chú ý __CODE_BLOCK_2__. Hoạt động này pha trộn các ngữ cảnh tương đồng giữa các từ.
- Bước 3 — Nhân với Value & Chiều đầu ra: Nhân trọng số chú ý với ma trận __MATH_BLOCK_5__ để thu được vector ngữ cảnh __CODE_BLOCK_3__. Cuối cùng, ghép (Concatenate) các heads con lại thành kích thước __CODE_BLOCK_4__ và nhân với ma trận chiều đầu ra __MATH_BLOCK_6__.

Sơ đồ luồng tính toán

```
graph TD; Input["Đầu vào Q, K, V  
[batch_size, seq_len, d_model]"] --> LQ["Linear Layer (W_q)"]; Input --> LK["Linear Layer (W_k)"]; Input --> LV["Linear Layer (W_v)"]; LQ --> Split["Phân tách thành nhiều Heads và d_k chiều  
(d_k = d_model / num_heads)"]; LK --> Split; LV --> Split; Split --> Attention["Scaled Dot-Product Attention (song song trên từng head)  
Attention(Q, K, V) = softmax(QK^T / sqrt(d_k)) * V"]; Attention --> Output["Gộp heads (Concat) + Chi chiều tuyến tính cuối (W_o)"];
```

* Sơ đồ luồng tính toán Multi-Head Attention

5.2. Khái niệm, Cách áp dụng và Ý nghĩa của Lớp Tuyến tính (Linear Layer) trong Transformer

5.2.1. Lớp Tuyến Tính (Linear Layer) là gì?

1. Định nghĩa đơn giản nhất

Lớp tuyến tính thực chất chỉ là một phép nhân ma trận rồi cộng thêm hệ số điều chỉnh:

$$y = x \cdot W^T + b$$

Trong đó:

- __MATH_BLOCK_0__: Vector đầu vào (ví dụ: vector nhúng của một từ).

- `__MATH_BLOCK_0__`: Ma trận trọng số (Weight) — đây là thứ mà mô hình tự học được trong quá trình huấn luyện.
- `__MATH_BLOCK_0__`: Vector thiên lệch (Bias) — hệ số điều chỉnh cộng thêm.
- `__MATH_BLOCK_0__`: Vector đầu ra sau khi biến đổi.

2. Ảnh dụ trực quan

Hãy tưởng tượng chúng ta có một tấm kính lọc màu:

- Ánh sáng trắng (đầu vào `__MATH_BLOCK_0__`) chiếu qua tấm kính.
- Tấm kính (ma trận trọng số `__MATH_BLOCK_0__`) lọc và pha trộn các thành phần màu.
- Ánh sáng ra phía bên kia (đầu ra `__MATH_BLOCK_0__`) có màu sắc hoàn toàn mới.

Mỗi lớp tuyến tính `__CODE_BLOCK_0__`, `__CODE_BLOCK_1__`, `__CODE_BLOCK_2__` trong Multi-Head Attention giống như 3 tấm kính lọc khác nhau, chiếu cùng một tín hiệu đầu vào thành 3 "góc nhìn" khác nhau (Query, Key, Value).

3. Ví dụ tính toán cụ thể bằng số

Giả sử đầu vào là vector `__MATH_BLOCK_0__` có `__MATH_BLOCK_1__` chiều và ta muốn chiếu ra `__MATH_BLOCK_2__` chiều (`__CODE_BLOCK_0__`):

$$x = [2, 1, 3, 0]$$

Ma trận trọng số `__MATH_BLOCK_0__` (kích thước `__MATH_BLOCK_1__`, mô hình tự học):

$$W = \begin{bmatrix} 0.5 & 0.2 & -0.1 & 0.3 \\ -0.3 & 0.7 & 0.4 & 0.1 \\ 0.1 & -0.5 & 0.6 & 0.8 \end{bmatrix}$$

Vector bias `__MATH_BLOCK_0__` (kích thước `__MATH_BLOCK_1__`):

$$b = [0.1, -0.2, 0.3]$$

Tính toán `__MATH_BLOCK_0__`:

$$y_1 = (2 \times 0.5) + (1 \times 0.2) + (3 \times -0.1) + (0 \times 0.3) + 0.1 = 1.0 + 0.2 - 0.3 + 0 + 0.1 = \mathbf{1.0}$$

$$y_2 = (2 \times -0.3) + (1 \times 0.7) + (3 \times 0.4) + (0 \times 0.1) - 0.2 = -0.6 + 0.7 + 1.2 + 0 - 0.2 = \mathbf{1.1}$$

$$y_3 = (2 \times 0.1) + (1 \times -0.5) + (3 \times 0.6) + (0 \times 0.8) + 0.3 = 0.2 - 0.5 + 1.8 + 0 + 0.3 = \mathbf{1.8}$$

$$y = [1.0, 1.1, 1.8]$$

4. Lớp tuyến tính cuối cùng

Lớp tuyến tính Generator đóng vai trò cực kỳ quan trọng ở bước cuối cùng của Decoder. Nhiệm vụ của nó là ánh xạ vector ẩn đầu ra của Decoder (kích thước `__MATH_BLOCK_0__`) sang không gian từ vựng có kích thước `__MATH_BLOCK_1__` chiều.

Ví dụ, sau khi đi qua Generator, ta thu được một vector điểm số (logits) kích thước `__MATH_BLOCK_0__`. Áp dụng hàm Softmax để tính xác suất cho từng từ trong từ điển:

Logits / Xác suất [18000 chiều]: [0.01, 0.002, 0.0001, ..., 0.15, ..., 0.001]

↑
Từ có xác suất cao nhất
→ Dự đoán từ tiếp theo

5. Tại sao gọi là "Tuyến tính" và Tại sao cần nó?

- Gọi là "tuyến tính" vì phép biến đổi `__MATH_BLOCK_0__` thỏa mãn tính chất tuyến tính: nếu nhân đầu vào lên `__MATH_BLOCK_1__` lần thì đầu ra cũng tăng tỷ lệ `__MATH_BLOCK_2__` lần (bỏ qua bias). Về mặt hình học, nó chỉ thực hiện các phép xoay, co giãn và dịch chuyển không gian vector mà không bẻ cong.
- Tại sao cần nó? Dữ liệu thô (vector nhúng từ) nằm trong một không gian vector "chung chung". Lớp tuyến tính đóng vai trò như một phép biến đổi không gian có thể học được, giúp chiếu dữ liệu sang một không gian mới phù hợp hơn cho từng nhiệm vụ cụ thể (tìm kiếm Query, đối khớp Key, truyền tải nội dung Value, dự đoán từ tiếp theo...).

6. Tổng kết các lớp tuyến tính trong Transformer

Lớp	Kích thước	Vai trò	Số trọng số cần học
<code>__CODE_BLOCK_0__</code>	<code>__MATH_BLOCK_0__</code>	Chiều thành Query	<code>__MATH_BLOCK_0__</code>
<code>__CODE_BLOCK_0__</code>	<code>__MATH_BLOCK_0__</code>	Chiều thành Key	<code>__MATH_BLOCK_0__</code>
<code>__CODE_BLOCK_0__</code>	<code>__MATH_BLOCK_0__</code>	Chiều thành Value	<code>__MATH_BLOCK_0__</code>
<code>__CODE_BLOCK_0__</code>	<code>__MATH_BLOCK_0__</code>	Kết hợp các heads	<code>__MATH_BLOCK_0__</code>
<code>__CODE_BLOCK_0__</code> (FFN)	<code>__MATH_BLOCK_0__</code>	Mở rộng không gian	<code>__MATH_BLOCK_0__</code>
<code>__CODE_BLOCK_0__</code> (FFN)	<code>__MATH_BLOCK_0__</code>	Thu hẹp không gian	<code>__MATH_BLOCK_0__</code>
Generator	<code>__MATH_BLOCK_0__</code>	Dự đoán từ tiếp theo	<code>__MATH_BLOCK_0__</code>

Riêng các lớp tuyến tính đã chiếm phần lớn tổng số tham số của toàn bộ mô hình Transformer. Đây chính là "bộ não" mà mô hình sử dụng để học cách biểu diễn và biến đổi ngôn ngữ.

6. Mạng Feed-Forward (FFN) và Hàm Kích hoạt

6.1. Kiến trúc mạng Position-Wise Feed-Forward (FFN)

Mạng Feed-Forward (FFN) được áp dụng đồng nhất cho mỗi vị trí của chuỗi đầu ra sau bước Self-Attention. Nó bao gồm hai phép biến đổi tuyến tính với một hàm kích hoạt phi tuyến ở giữa:

$$FFN(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

Trong đó, phương trình trên sử dụng hàm kích hoạt ReLU.

6.2. Tại sao lại dùng ReLU cho FFN và so sánh các hàm kích hoạt

6.2.1. ReLU là gì?

ReLU (Rectified Linear Unit) là hàm kích hoạt cực kỳ đơn giản:

$$ReLU(x) = \max(0, x)$$

- Nếu `__MATH_BLOCK_0__` → giữ nguyên `__MATH_BLOCK_1__`.
- Nếu `__MATH_BLOCK_0__` → trả về `__MATH_BLOCK_1__`.

Ảnh dụ dễ hiểu: ReLU giống như một cánh cổng một chiều. Tín hiệu dương (thông tin hữu ích) được cho qua tự do, còn tín hiệu âm (nhiều) bị chặn hoàn toàn.

6.2.2. Tại sao FFN cần hàm kích hoạt (phi tuyến)?

Lớp Attention bản chất là phép nhân ma trận + Softmax — đây chủ yếu là các phép tuyến tính. Nếu FFN cũng chỉ gồm 2 lớp tuyến tính chồng lên nhau mà không có hàm kích hoạt:

$$FFN(x) = xW_1W_2 + b$$

Theo đại số tuyến tính, tích của 2 ma trận vẫn là 1 ma trận: `__MATH_BLOCK_0__`. Toàn bộ FFN sẽ sụp đổ thành 1 lớp tuyến tính duy nhất, mất hoàn toàn khả năng học các mối quan hệ phức tạp.

Hàm ReLU chen vào giữa để bẻ gãy tính tuyến tính, cho phép mô hình học được các biên quyết định phi tuyến.

6.2.3. Tại sao chọn ReLU mà không phải hàm khác?

Có nhiều hàm kích hoạt khác như Sigmoid, Tanh, GELU, SiLU... Bài báo gốc năm 2017 chọn ReLU vì 4 lý do chính:

6.2.3.1. Tốc độ tính toán cực nhanh

Hàm	Công thức	Phép toán
ReLU	<code>__MATH_BLOCK_0__</code>	Chỉ cần 1 phép so sánh
Sigmoid	<code>__MATH_BLOCK_0__</code>	Cần tính <code>__MATH_BLOCK_0__</code> , phép chia, phép cộng
Tanh	<code>__MATH_BLOCK_0__</code>	Cần tính 2 lần hàm mũ, phép trừ, phép chia
GELU	<code>__MATH_BLOCK_0__</code>	Cần tính hàm phân phối chuẩn tích lũy

Trong FFN của Transformer, phép tính ReLU được áp dụng trên vector 2048 chiều cho mỗi từ, mỗi lớp, mỗi batch. Ví dụ: batch 32 câu × 50 từ × 6 lớp = 9,600 lần gọi ReLU.

- Tính toán cực nhanh: Phép tính ReLU chỉ đơn giản là so sánh một số với `__MATH_BLOCK_0__`. So với các hàm phi tuyến truyền thống như Sigmoid hay Tanh đòi hỏi nhiều phép tính hàm mũ `__MATH_BLOCK_1__` và phép chia phức tạp, ReLU giúp giảm đáng kể thời gian tính toán lan truyền xuôi và lan truyền ngược trên GPU.
- Hạn chế triệt tiêu gradient (Vanishing Gradient): Với các giá trị dương (`__MATH_BLOCK_0__`), đạo hàm của ReLU luôn luôn bằng `__MATH_BLOCK_1__`. Điều này giúp dòng gradient truyền trực tiếp qua các lớp ẩn mà không bị thu nhỏ dần như khi đi qua hàm Sigmoid hay Tanh (vốn có đạo hàm tiệm cận về 0 khi giá trị đầu vào lớn).
- Tạo ra biểu diễn thưa (Sparse Representation): Với các giá trị đầu vào âm (`__MATH_BLOCK_0__`), đầu ra của ReLU bằng `__MATH_BLOCK_1__`. Điều này có nghĩa là tại mỗi thời điểm, chỉ có một phần các nơ-ron được kích hoạt hoạt động. Tính thưa này mô phỏng hành vi của não bộ con người và giúp mô hình lưu trữ các đặc trưng thông tin độc lập tốt hơn.

ReLU là hàm kích hoạt chuẩn công nghiệp được chứng minh ổn định trên hàng trăm bài báo (ResNet, Inception, VGG...). Các tác giả Transformer ưu tiên sự đơn giản và đáng tin cậy thay vì thử nghiệm hàm mới chưa được kiểm chứng kỹ.

6.2.4. Các mô hình hiện đại đã thay thế ReLU bằng gì?

Sau năm 2017, cộng đồng nghiên cứu đã tìm ra các hàm kích hoạt tốt hơn ReLU cho Transformer:

Mô hình	Hàm kích hoạt	Công thức	Ưu điểm so với ReLU
GPT-2, BERT	GELU	<code>__MATH_BLOCK_0__</code>	Cho gradient mượt hơn ở vùng <code>__MATH_BLOCK_0__</code> , không bị "chết nơ-ron"
LLaMA, PaLM	SiLU (Swish)	<code>__MATH_BLOCK_0__</code>	Tương tự GELU, đơn giản hơn
LLaMA 2/3	SwiGLU	<code>__MATH_BLOCK_0__</code>	Thêm cơ chế "cổng" kiểm soát dòng thông tin

Điểm yếu duy nhất của ReLU mà các hàm trên khắc phục: Hiện tượng "Dying ReLU" — khi một nơ-ron nhận đầu vào âm liên tục trong quá trình huấn luyện, nó bị tắt vĩnh viễn (`__MATH_BLOCK_0__`, `__MATH_BLOCK_1__`) và không bao giờ được cập nhật lại. GELU/SiLU cho phép gradient nhỏ (nhưng khác 0) ở vùng âm, giúp nơ-ron có cơ hội "hồi sinh".

6.2.5. Tóm lại: Tại sao dự án của chúng ta dùng ReLU?

Dự án của chúng ta cài đặt trung thành theo đúng bài báo gốc "Attention Is All You Need" (2017). Tại thời điểm đó, ReLU là lựa chọn tối ưu nhất vì:

- 1. Nhanh — chỉ 1 phép so sánh.
- 2. Gradient ổn định — không bị triệt tiêu.
- 3. Tính thưa — giúp mô hình biểu diễn đa dạng.
- 4. Đơn giản, đáng tin cậy — đã được kiểm chứng rộng rãi.

Nếu muốn cải tiến thêm, chúng ta có thể thay `__CODE_BLOCK_0__` bằng `__CODE_BLOCK_1__` trong lớp `__CODE_BLOCK_2__` để đạt hiệu năng tốt hơn (giống GPT/BERT hiện đại).

7. Residual Connection & Layer Normalization (Kết nối tắt & Chuẩn hóa lớp)

7.1. Khái niệm và Vai trò của Residual Connection & Layer Normalization

7.1.1. Residual Connection (Kết nối tắt)

1. Vấn đề cần giải quyết: Suy giảm gradient

Khi xếp chồng nhiều lớp mạng nơ-ron lên nhau (ví dụ 6 lớp Encoder), tín hiệu đầu vào phải đi qua rất nhiều phép biến đổi liên tiếp:

Đầu vào $x \rightarrow$ [Lớp 1] $\rightarrow$ [Lớp 2] $\rightarrow$ [Lớp 3] $\rightarrow$ [Lớp 4] $\rightarrow$ [Lớp 5] $\rightarrow$ [Lớp 6] $\rightarrow$ Đầu ra

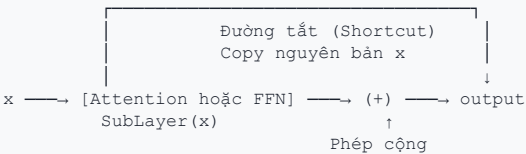
Trong quá trình huấn luyện, gradient (tín hiệu cập nhật trọng số) phải truyền ngược từ lớp 6 về lớp 1. Mỗi khi đi qua một lớp, gradient bị nhân với đạo hàm của lớp đó. Sau nhiều lần nhân liên tiếp:

- Nếu đạo hàm < 1 : gradient teo dần về 0 $\rightarrow$ Các lớp đầu không học được gì (Vanishing Gradient).
- Nếu đạo hàm > 1 : gradient bùng nổ $\rightarrow$ Trọng số nhảy loạn xạ (Exploding Gradient).

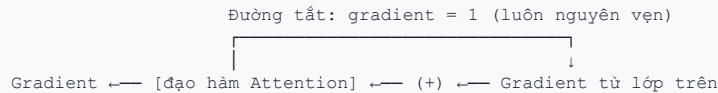
2. Giải pháp: Tạo "đường cao tốc" cho gradient

Residual Connection cộng trực tiếp đầu vào `__MATH_BLOCK_0__` vào đầu ra của khối con:

$$\text{output} = x + \text{SubLayer}(x)$$



Gradient giờ có 2 con đường để truyền ngược:



Nhờ có đường cộng (+), gradient khi truyền ngược từ lớp trên xuống sẽ được phân tách thành hai nhánh: một nhánh đi qua lớp con SubLayer (như Self-Attention hay Feed-Forward) và một nhánh đi trực tiếp qua đường tắt Residual. Ở nhánh đường tắt, gradient được nhân với đạo hàm của phép cộng (bằng `__MATH_BLOCK_0__`), nghĩa là gradient được bảo toàn nguyên vẹn và truyền thẳng về các lớp phía trước. Điều này ngăn chặn hiện tượng triệt tiêu gradient một cách hiệu quả, cho phép mô hình Transformer có thể huấn luyện sâu tới hàng trăm lớp mà vẫn hội tụ tốt.

7.1.2. Ví dụ cụ thể bằng số về Residual Connection & Layer Normalization

Để làm rõ cách hoạt động của khối kết nối Pre-LN, ta xét một ví dụ số học đơn giản với đầu vào `__MATH_BLOCK_0__` là một vector `__MATH_BLOCK_1__` chiều:

```
x = [4.0, 8.0, 2.0, 6.0]
```

Bước 1: Khối Self-Attention với Pre-LN

(A) Chuẩn hóa lớp (LayerNorm) trước khi đưa vào Attention:

Ta chuẩn hóa vector `__MATH_BLOCK_0__` về trung bình bằng `__MATH_BLOCK_1__` và phương sai bằng `__MATH_BLOCK_2__` trước khi đưa vào Attention

```
attn_out = Attention(norm_x) = [0.12, -0.30, 0.25, 0.08]
→ Pha trộn ngữ cảnh từ các từ khác
```

```
(C) Cộng Residual (đường tắt):
x_new = x + dropout(attn_out)
       = [4.0+0.12, 8.0+(-0.30), 2.0+0.25, 6.0+0.08]
       = [4.12,      7.70,      2.25,      6.08]
→ BẢO TOÀN thông tin gốc x, chỉ BỔ SUNG ngữ cảnh mới
```

== KHỐI CON 2: FEED-FORWARD ==

```
(D) LayerNorm chuẩn hóa trước:
norm_x = LayerNorm(x_new) = [-0.51, 1.30, -1.28, 0.49]
→ Ổn định hóa lần 2
```

```
(E) Feed-Forward xử lý:
ff_out = FFN(norm_x) = [0.05, 0.15, -0.10, 0.20]
→ Bổ sung biểu diễn phi tuyến
```

```
(F) Cộng Residual (đường tắt):
x_final = x_new + dropout(ff_out)
         = [4.12+0.05, 7.70+0.15, 2.25+(-0.10), 6.08+0.20]
         = [4.17,      7.85,      2.15,      6.28]
→ Thông tin gốc vẫn được bảo toàn xuyên suốt!
```

```
Đầu ra = [4.17, 7.85, 2.15, 6.28]
```

7.1.3. Tóm tắt tác dụng của từng kỹ thuật

Kỹ thuật	Vấn đề giải quyết	Cách hoạt động	Lợi ích
Residual Connection	Gradient bị triệt tiêu khi mạng sâu	Cộng thẳng đầu vào vào đầu ra: <code>__MATH_BLOCK_0__</code>	Gradient luôn có đường truyền nguyên vẹn (đạo hàm = 1). Thông tin gốc được bảo toàn qua mọi lớp
Layer Normalization	Giá trị kích hoạt bùng nổ hoặc lệch sau nhiều lớp	Kéo phân phối về trung bình = 0, phương sai = 1	Ổn định hóa tín hiệu đầu vào cho Softmax và các phép biến đổi tiếp theo. Huấn luyện hội tụ nhanh hơn
Kết hợp cả hai (Pre-LN)	Huấn luyện Transformer sâu (6+ lớp)	LayerNorm trước → SubLayer → Cộng Residual	Mô hình hội tụ ổn định ngay từ đầu, không cần warmup phức tạp. Cho phép xếp chồng nhiều lớp mà không lo gradient

7.2. Phân tích So sánh Pre-LN vs Post-LN

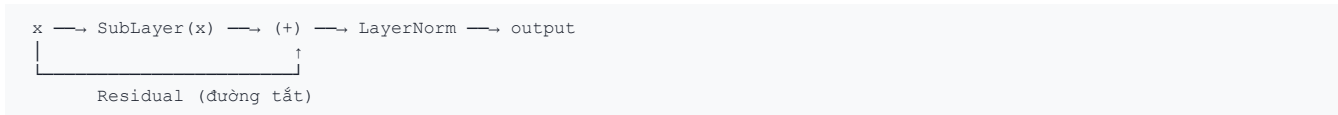
7.2.1. Sự khác biệt cốt lõi (Pre-LN vs Post-LN)

Cả hai kiến trúc đều sử dụng đúng 3 thành phần giống nhau: LayerNorm, SubLayer (Attention hoặc FFN), và Residual Connection. Điểm khác biệt duy nhất là vị trí đặt LayerNorm:

Post-LN (Bài báo gốc 2017)

output = LayerNorm(x + SubLayer(x))

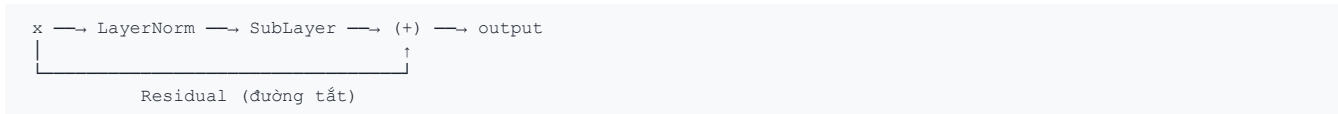
LayerNorm nằm sau phép cộng Residual:



Pre-LN (Dự án của chúng ta, GPT, LLaMA)

output = x + SubLayer(LayerNorm(x))

LayerNorm nằm trước khi vào SubLayer:



7.2.2. Tại sao vị trí LayerNorm lại quan trọng?

Sự khác biệt tưởng chừng nhỏ này lại ảnh hưởng cực lớn đến luồng gradient khi truyền ngược. Hãy xem xét gradient truyền qua 6 lớp Encoder:

Post-LN: Gradient bị "pha loãng" qua từng lớp

Lớp 6: gradient truyền về

- ↓ đi qua LayerNorm → gradient bị chia cho σ
- ↓ chia thành 2 nhánh (residual + sublayer)

Lớp 5: gradient đã yếu hơn

- ↓ đi qua LayerNorm → gradient bị chia cho σ lần nữa
- ↓ chia thành 2 nhánh

Lớp 4: gradient yếu hơn nữa

- ↓ ...

Lớp 1: gradient gần như bằng 0 ← KHÔNG HỌC ĐƯỢC GÌ!

Vấn đề: LayerNorm nằm trên đường tắt Residual. Gradient muốn đi qua đường cao tốc (Residual) cũng bị ép phải đi qua LayerNorm. Mỗi lần đi qua LayerNorm, gradient sẽ bị biến đổi và chia cho độ lệch chuẩn của lớp hiện tại. Khi số lớp tăng lên, các phép chuẩn hóa liên tiếp này sẽ bóp nghẹt dòng gradient truyền ngược, khiến nó bị triệt tiêu gần như hoàn toàn khi về tới các lớp đầu tiên. Do đó, để huấn luyện Post-LN thành công, người ta bắt buộc phải sử dụng giai đoạn khởi động (warmup) với tốc độ học cực nhỏ để tránh làm hỏng mô hình lúc đầu.

Ngược lại, với Pre-LN, LayerNorm được đặt trước các khối chú ý và mạng feed-forward. Lúc này, đường dẫn Residual đóng vai trò là một "đường cao tốc truyền gradient" không hề bị cản trở bởi bất kỳ phép chuẩn hóa nào, giúp gradient chảy tự do từ lớp cuối cùng về lớp đầu tiên mà không bị triệt tiêu.

7.2.3. So sánh trực quan qua Code PyTorch

5.1. Cài đặt Pre-LN (Mô hình của chúng ta)

```
def forward(self, x, mask):
    # 1. Chuẩn hóa TRƯỚC khi đưa vào Attention
    norm_x = self.norm1(x)
    attn_out, _ = self.self_attention(norm_x, norm_x, norm_x, mask)
    x = x + self.dropout(attn_out) # Cộng thẳng vào x gốc

    # 2. Chuẩn hóa TRƯỚC khi đưa vào Feed-Forward
    norm_x2 = self.norm2(x)
    ff_out = self.feed_forward(norm_x2)
    x = x + self.dropout(ff_out) # Cộng thẳng vào x gốc
    return x
```

5.2. Cài đặt Post-LN (Bài báo gốc)

```
def forward(self, x, mask):
    # 1. Đi qua Attention trước, cộng residual rồi mới chuẩn hóa
    attn_out, _ = self.self_attention(x, x, x, mask)
    x = self.norm1(x + self.dropout(attn_out)) # LayerNorm SAU phép cộng

    # 2. Đi qua Feed-Forward trước, cộng residual rồi mới chuẩn hóa
    ff_out = self.feed_forward(x) # Không LayerNorm trước

    # 2. Đi qua Feed-Forward trước, cộng residual rồi mới chuẩn hóa
    ff_out = self.feed_forward(x) # Không LayerNorm trước

    ff_out = self.feed_forward(x) # Không LayerNorm trước
    x = self.norm2(x + self.dropout(ff_out)) # LayerNorm SAU phép cộng
    return x
```

Sự khác biệt trong code chỉ là hoán đổi vị trí 1 dòng `__CODE_BLOCK_0__`, nhưng ảnh hưởng đến toàn bộ quá trình huấn luyện.

7.2.4. Lưu ý quan trọng: Final LayerNorm trong Pre-LN

Khi dùng Pre-LN, đầu ra của lớp Encoder/Decoder cuối cùng chưa được chuẩn hóa (vì LayerNorm chỉ nằm ở đầu vào mỗi sublayer, không phải đầu ra). Do đó, code của chúng ta có thêm một lớp LayerNorm cuối cùng ở cuối toàn bộ stack:

PHẦN III: LẮP GHÉP MÔ HÌNH & TỐI ƯU HÓA DỮ LIỆU

8. Lớp Encoder, Decoder và Kiến trúc Transformer Hoàn chỉnh

8.1. Lớp Encoder (Bộ mã hóa)

```
class Encoder(nn.Module):
    def __init__(self, vocab_size, d_model, num_layers, num_heads, d_ff,
                 max_len=5000, dropout=0.1, embedding=None):
        super(Encoder, self).__init__()
        self.d_model = d_model
        if embedding is not None:
            self.embedding = embedding
        else:
            self.embedding = nn.Embedding(vocab_size, d_model)
        self.pos_encoding = PositionalEncoding(d_model, max_len)
        self.layers = nn.ModuleList([
            EncoderLayer(d_model, num_heads, d_ff, dropout)
            for _ in range(num_layers)
        ])
        self.norm = nn.LayerNorm(d_model) # ← Final LayerNorm cho Pre-LN
        self.dropout = nn.Dropout(dropout)

    def forward(self, src, mask=None):
        x = self.embedding(src) * math.sqrt(self.d_model)
        x = self.pos_encoding(x)
        x = self.dropout(x)
        attn_list = []
        for layer in self.layers:
            x, attn_weights = layer(x, mask)
            attn_list.append(attn_weights)
        return self.norm(x), attn_list
```

8.2. Lớp Decoder (Bộ giải mã)

Lớp `__CODE_BLOCK_0__` xếp chồng nhiều lớp `__CODE_BLOCK_1__` để xử lý chuỗi đích kết hợp với thông tin từ bộ mã hóa.

```
class Decoder(nn.Module):
    def __init__(self, vocab_size, d_model, num_layers, num_heads, d_ff,
                 max_len=5000, dropout=0.1, embedding=None):
        super(Decoder, self).__init__()
        self.d_model = d_model

        if embedding is not None:
            self.embedding = embedding
        else:
            self.embedding = nn.Embedding(vocab_size, d_model)

        self.pos_encoding = PositionalEncoding(d_model, max_len)
        self.layers = nn.ModuleList([
            DecoderLayer(d_model, num_heads, d_ff, dropout)
            for _ in range(num_layers)
        ])
        self.norm = nn.LayerNorm(d_model)
        self.dropout = nn.Dropout(dropout)

    def forward(self, tgt, enc_output, src_mask=None, tgt_mask=None):
        x = self.embedding(tgt) * math.sqrt(self.d_model)
        x = self.pos_encoding(x)
        x = self.dropout(x)

        self_attn_list, cross_attn_list = [], []
        for layer in self.layers:
            x, self_attn, cross_attn = layer(x, enc_output, src_mask, tgt_mask)
            self_attn_list.append(self_attn)
            cross_attn_list.append(cross_attn)
        return self.norm(x), self_attn_list, cross_attn_list
```

Các thành phần chính của Decoder:

- `__CODE_BLOCK_0__`: Lớp nhúng của chuỗi đích. Có thể dùng chung lớp nhúng từ Encoder nếu áp dụng Shared Embedding.
- `__CODE_BLOCK_0__`: Danh sách `__CODE_BLOCK_1__` lớp `__CODE_BLOCK_2__` hoạt động tuần tự.
- `__CODE_BLOCK_0__`: Lớp chuẩn hóa cuối cùng (Final LayerNorm) của bộ giải mã.

8.3. Lớp Transformer và Cơ chế Weight Tying

Lớp `__CODE_BLOCK_0__` kết nối cả hai khối `__CODE_BLOCK_1__` và `__CODE_BLOCK_2__` và áp dụng cơ chế chia sẻ trọng số (Weight Tying).

```
class Transformer(nn.Module):
    def __init__(self, src_vocab_size, tgt_vocab_size, d_model=512, num_layers=6, num_heads=8, d_ff=2048,
                 max_len=5000, dropout=0.1):
        super(Transformer, self).__init__()

        # 1. Khởi tạo một Embedding dùng chung nếu kích thước từ điển bằng nhau
        if src_vocab_size == tgt_vocab_size:
            self.shared_embedding = nn.Embedding(src_vocab_size, d_model)
            self.encoder = Encoder(src_vocab_size, d_model, num_layers, num_heads, d_ff,
                                  max_len=max_len, dropout=dropout, embedding=self.shared_embedding)
            self.decoder = Decoder(tgt_vocab_size, d_model, num_layers, num_heads, d_ff,
                                  max_len=max_len, dropout=dropout, embedding=self.shared_embedding)
        else:
            self.encoder = Encoder(src_vocab_size, d_model, num_layers, num_heads, d_ff, max_len=max_len,
                                  dropout=dropout)
            self.decoder = Decoder(tgt_vocab_size, d_model, num_layers, num_heads, d_ff, max_len=max_len,
                                  dropout=dropout)

        # 2. Định nghĩa lớp chiếu tuyến tính đầu ra (Generator)
        self.generator = nn.Linear(d_model, tgt_vocab_size)

        # 3. Áp dụng Weight Tying giữa Decoder Embedding và Generator
        if src_vocab_size == tgt_vocab_size:
            self.generator.weight = self.shared_embedding.weight
        else:
            self.generator.weight = self.decoder.embedding.weight

    def forward(self, src, tgt, src_mask=None, tgt_mask=None):
        # Bộ mã hóa xử lý câu nguồn
        enc_output, enc_attn = self.encoder(src, src_mask)
        # Bộ giải mã sinh chuỗi đầu ra kết hợp thông tin chú ý chéo
        dec_output, dec_self_attn, dec_cross_attn = self.decoder(tgt, enc_output, src_mask, tgt_mask)
        # Chuyển sang kích thước từ điển để tính logits
        logits = self.generator(dec_output)
        return logits, enc_attn, dec_self_attn, dec_cross_attn
```

8.4. Sơ đồ mô tả Dòng chảy Dữ liệu toàn diện (Full Data Flow)

Dưới đây là sơ đồ chi tiết biểu diễn luồng dữ liệu đi qua toàn bộ mạng Transformer từ từ nguồn tiếng Việt sang từ đích tiếng Anh:

=== BƯỚC 1: ENCODER ===

```
enc_output = Encoder(src)
Token ID [45, 120, 88]
  ↓ Shared Embedding (tra bảng)
Vector thô [v□□, v□□□, v□□]
  ↓ × √512 + PE + Dropout
  ↓ 6 Encoder Layers (Self-Attention + FFN)
  ↓ Final LayerNorm
enc_output = [v_tôi_ctx, v_yêu_ctx, v_học_ctx]      [1, 3, 512]
(Mỗi vector chứa ngữ cảnh toàn bộ câu nguồn)
```

=== BƯỚC 2: DECODER ===

```
dec_output = Decoder(tgt, enc_output)
Token ID [1, 205, 67]
  ↓ Shared Embedding (CÙNG ma trận với Encoder!)
Vector thô [v□, v□□□, v□□]
  ↓ × √512 + PE + Dropout
  ↓ 6 Decoder Layers:
  |   Masked Self-Attention (không nhìn từ tương lai)
  |   Cross-Attention (hỏi Encoder lấy ngữ cảnh "chúng ta yêu học")
  |   FFN
  ↓ Final LayerNorm
dec_output = [v_<sos>_ctx, v_we_ctx, v_learn_ctx]  [1, 3, 512]
```

=== BƯỚC 3: GENERATOR ===

```
logits = Generator(dec_output)
dec_output [1, 3, 512]
  ↓ × shared_embedding.weight^T      (CÙNG ma trận với Embedding!)
  ↓ (Đo tương đồng với vector nhúng của 18000 từ)
logits = [1, 3, 18000]

logits[0] = [2.1, 0.3, -1.5, ..., 5.8, ...] ← Điểm số cho 18000 từ tại vị trí 0
logits[1] = [0.5, 1.2, -0.8, ..., 4.3, ...] ← Điểm số cho 18000 từ tại vị trí 1
logits[2] = [1.0, 0.1, -2.0, ..., 6.1, ...] ← Điểm số cho 18000 từ tại vị trí 2
                                     ↑
                               Từ có điểm cao nhất
                               → Dự đoán từ tiếp theo
```

=== ĐẦU RA ===

```
return (logits, enc_attn, dec_self_attn, dec_cross_attn)
logits:      [1, 3, 18000] → Dùng tính Loss và dự đoán
enc_attn:    6 ma trận    → Vẽ Encoder Attention Map
dec_self_attn: 6 ma trận    → Vẽ Decoder Self-Attention Map
dec_cross_attn: 6 ma trận    → Vẽ Cross-Attention Map (quan trọng nhất!)
```

9. Xử lý Dữ liệu & Cơ chế BPE Tokenizer và Đệm động (Dynamic Padding)

9.1. Cơ chế BPE Tokenizer và Xử lý dữ liệu lớn

9.1.1. Vấn đề với Tokenizer thông thường (Word-level)

Tokenizer thông thường tách từ theo khoảng trắng. Mỗi từ riêng biệt là một token:

```
Câu:      "chúng ta yêu học máy"
Tokens:   ["chúng ta", "yêu", "học", "máy"]
Token ID: [45,      120,    88,    203]
```

3 vấn đề nghiêm trọng:

Vấn đề	Giải thích	Ví dụ
Từ điển khổng lồ	Phải lưu mọi từ đã gặp → hàng trăm nghìn từ	"learning", "learned", "learns" → 3 mục riêng biệt
Từ lạ (OOV)	Gặp từ chưa bao giờ thấy → gán <code>__CODE_BLOCK_0__</code> → mất thông tin	"blockchain" → <code>__CODE_BLOCK_0__</code> (không biết nghĩa)
Lãng phí	Các từ có cùng gốc nhưng mô hình coi là hoàn toàn khác nhau	"học", "học_sinh", "học_viên" → 3 vector độc lập

9.1.2. BPE (Byte Pair Encoding) giải quyết thế nào?

BPE không tách theo từ, mà tách theo cụm ký tự con (subwords) dựa trên tần suất xuất hiện.

Thuật toán BPE hoạt động theo 3 bước:

Bước 1: Bắt đầu từ từng ký tự đơn lẻ

```
"learning" → ["l", "e", "a", "r", "n", "i", "n", "g"]
```

Bước 2: Đếm cặp ký tự liền kề xuất hiện nhiều nhất và gộp lại

```
Vòng 1: Cặp ("l","e") xuất hiện nhiều nhất → gộp thành "le"
"learning" → ["le", "a", "r", "n", "i", "n", "g"]

Vòng 2: Cặp ("le","a") xuất hiện nhiều nhất → gộp thành "lea"
"learning" → ["lea", "r", "n", "i", "n", "g"]

Vòng 3: Cặp ("lea","r") → gộp thành "lear"
"learning" → ["lear", "n", "i", "n", "g"]

Vòng 4: Cặp ("n","i") → gộp thành "ni"
"learning" → ["lear", "ni", "n", "g"]

Vòng 5: Cặp ("ni","n") → gộp thành "nin"
"learning" → ["lear", "nin", "g"]

Vòng 6: Cặp ("nin","g") → gộp thành "ning"
"learning" → ["lear", "ning"]
```

Bước 3: Lặp lại quá trình này cho đến khi đạt kích thước từ vựng tối đa (ví dụ: 18000 subwords).

9.1.3. Cài đặt BPE Tokenizer trong codebase của chúng ta

Chúng ta huấn luyện một BPE Tokenizer chung cho cả hai ngôn ngữ (Shared Vocab) từ thư mục `__CODE_BLOCK_0__` để tối ưu hóa bộ nhớ:

```
from tokenizers import Tokenizer
from tokenizers.models import BPE
from tokenizers.trainers import BpeTrainer
from tokenizers.pre_tokenizers import Whitespace

# Khởi tạo Tokenizer BPE
tokenizer = Tokenizer(BPE(unk_token="[UNK]"))
tokenizer.pre_tokenizer = Whitespace()

# Khởi tạo trainer với các token đặc biệt
trainer = BpeTrainer(
    special_tokens=["[PAD]", "[UNK]", "[SOS]", "[EOS]"],
    vocab_size=18000
)

# Huấn luyện trên dữ liệu
tokenizer.train_from_iterator(corpus, trainer)
```

Khi giải mã và mã hóa câu:

```
# Ví dụ mã hóa câu
src_sent = "tôi học deep learning ."
tgt_sent = "i learn deep learning ."

print(shared_vocab.encode(src_sent, add_special=False))
print(shared_vocab.encode(tgt_sent, add_special=True)) # Đích: CÓ
```

- Câu nguồn (Encoder): Encoder chỉ cần đọc hiểu nội dung câu nguồn. Nó không cần biết đâu là "bắt đầu" hay "kết thúc" vì nó xử lý toàn bộ câu cùng lúc.
- Câu đích (Decoder): Decoder sinh từ từng từ một theo cơ chế tự hồi quy:
- `__CODE_BLOCK_0__` (Start of Sentence): Tín hiệu khởi động. Decoder nhận `__CODE_BLOCK_1__` và bắt đầu sinh từ đầu tiên.
- `__CODE_BLOCK_0__` (End of Sentence): Tín hiệu dừng. Khi Decoder sinh ra `__CODE_BLOCK_1__`, quá trình dịch kết thúc.

```
Khi huấn luyện:
tgt_input = [<sos>, i, love, deep, learning]      ← Đầu vào Decoder (bỏ <eos>)
tgt_target = [i, love, deep, learning, <eos>]      ← Nhãn đúng (bỏ <sos>)

Decoder nhận <sos>      → phải dự đoán "i"
Decoder nhận i          → phải dự đoán "love"
Decoder nhận love       → phải dự đoán "deep"
Decoder nhận deep       → phải dự đoán "learning"
Decoder nhận learning    → phải dự đoán <eos> (dừng)
```

9.1.4. Tóm tắt so sánh Tokenizer

	Word-level Tokenizer	BPE Tokenizer
Cách tách	Theo khoảng trắng	Theo cụm ký tự con (subwords)
Kích thước từ điển	50,000+ từ	Chỉ 18,000 subwords
Từ lạ (OOV)	Gán <code>__CODE_BLOCK_0__</code> → mất thông tin	Tách thành subwords → KHÔNG BAO GIỜ <code>__CODE_BLOCK_0__</code>
Chia sẻ gốc từ	Không ("học", "học_sinh" là 2 từ riêng)	Có ("học" là subword chung)
Bộ nhớ Embedding	50000×512 = 25.6M tham số	18000×512 = 9.2M tham số

So sánh Hiệu năng Đệm dữ liệu	Minh họa Đệm động (Dynamic Padding)																								
So sánh cơ chế đệm dữ liệu đầu vào giữa Static Padding (cố định) và Dynamic Padding (động): • Static Padding (Đệm cố định): Gán cứng chiều dài tối đa (ví dụ 100 token) cho tất cả các batch. Gây lãng phí VRAM cực lớn (50-80% ô trống <code>__CODE_BLOCK_0__</code>), làm chậm quá trình tính toán trên GPU do phải xử lý các token đệm vô nghĩa. • Dynamic Padding (Đệm động): Tự động đệm các câu trong từng batch theo độ dài của câu dài nhất chỉ trong batch đó. Tối ưu hóa bộ nhớ GPU (lãng phí chỉ 5-20%), tăng tốc độ huấn luyện từ 2 đến 3 lần.	STATIC PADDING (Đệm cố định max_len = 8) <table><tr><td>tôi</td><td>yêu</td><td>học</td><td>máy</td><td><pad></td><td><pad></td><td><pad></td><td><pad></td></tr><tr><td>bạn</td><td>khỏe</td><td>không</td><td><pad></td><td><pad></td><td><pad></td><td><pad></td><td><pad></td></tr></table> DYNAMIC PADDING (Đệm động theo batch - max_len_batch = 4) <table><tr><td>tôi</td><td>yêu</td><td>học</td><td>máy</td></tr><tr><td>bạn</td><td>khỏe</td><td>không</td><td><pad></td></tr></table> * Sơ đồ minh họa cơ chế Dynamic Padding	tôi	yêu	học	máy	<pad>	<pad>	<pad>	<pad>	bạn	khỏe	không	<pad>	<pad>	<pad>	<pad>	<pad>	tôi	yêu	học	máy	bạn	khỏe	không	<pad>
tôi	yêu	học	máy	<pad>	<pad>	<pad>	<pad>																		
bạn	khỏe	không	<pad>	<pad>	<pad>	<pad>	<pad>																		
tôi	yêu	học	máy																						
bạn	khỏe	không	<pad>																						

10. Huấn luyện Song song (Multi-GPU) & Thiết kế Mask tương thích (Device-Aware Masking)

10.1. Vấn đề khi chạy Multi-GPU & Thiết kế Mask tương thích

Khi bọc mô hình bằng `__CODE_BLOCK_0__`, PyTorch tự động chia đôi batch và gửi sang 2 GPU khác nhau:

```
Batch gốc: 32 câu
  | nn.DataParallel tự động chia
GPU 0 (cuda:0): 16 câu đầu
GPU 1 (cuda:1): 16 câu sau
```

Vấn đề: Nếu ta tạo mask trên một device cố định (ví dụ `__CODE_BLOCK_0__`), khi GPU 1 cần mask, nó sẽ bị lỗi:

```
LỖI: RuntimeError: Expected all tensors to be on the same device,
      but found at least two devices, cuda:0 and cuda:1!
```

Nguyên nhân:
GPU 1 đang xử lý dữ liệu trên cuda:1
Nhưng mask được tạo trên cuda:0
→ PyTorch không thể tính toán giữa 2 device khác nhau!

10.1.1. Giải pháp: Tạo mask trên cùng device với dữ liệu

```
def make_src_mask(src, pad_idx=0):
    src_mask = (src != pad_idx).unsqueeze(1).unsqueeze(2)
```

- `__CODE_BLOCK_0__`: Trả về một tensor Boolean có kích thước `__CODE_BLOCK_1__`, nơi các vị trí không phải `__CODE_BLOCK_2__` có giá trị `__CODE_BLOCK_3__`, và các vị trí `__CODE_BLOCK_4__` có giá trị `__CODE_BLOCK_5__`.
- `__CODE_BLOCK_0__`: Thêm hai chiều mới vào ma trận, chuyển kích thước thành `__CODE_BLOCK_1__`. Kích thước này tương thích để PyTorch thực hiện broadcasting tự động với ma trận điểm số attention của Multi-Head Attention (có hình dạng `__CODE_BLOCK_2__`).

Look-Ahead Mask (`__CODE_BLOCK_0__`):

```
tgt_pad_mask = (tgt != pad_idx).unsqueeze(1).unsqueeze(2)
seq_len = tgt.size(1)
no_peak_mask = torch.tril(torch.ones((seq_len, seq_len), device=tgt.device)).bool()
no_peak_mask = no_peak_mask.unsqueeze(0).unsqueeze(1)
tgt_mask = tgt_pad_mask & no_peak_mask
```

- `__CODE_BLOCK_0__` (Triangular Lower): Tạo ma trận tam giác dưới kích thước `__CODE_BLOCK_1__`, trong đó các phần tử ở đường chéo chính trở xuống có giá trị `__CODE_BLOCK_2__` (hoặc `__CODE_BLOCK_3__`), và các phần tử phía trên bằng `__CODE_BLOCK_4__` (hoặc `__CODE_BLOCK_5__`).
- Phép toán logic `__CODE_BLOCK_0__` (AND) kết hợp `__CODE_BLOCK_1__` and `__CODE_BLOCK_2__` để tạo ra mặt nạ cuối cùng cho Decoder. Nó đảm bảo mô hình không chú ý đến các token `__CODE_BLOCK_3__` VÀ không "nhìn trộm" các từ tương lai khi sinh từ hiện tại.

10.2. Vòng lặp huấn luyện (Training Loop) trong PyTorch

Vòng lặp huấn luyện là nơi mô hình thực hiện các bước lan truyền xuôi, tính loss, lan truyền ngược và cập nhật trọng số. Để huấn luyện mô hình một cách tối ưu trên Multi-GPU, chúng ta cấu hình vòng lặp như sau:

```

model.train()
epoch_loss = 0
for batch_idx, batch in enumerate(train_loader):
    src = batch['src'].to(device)
    tgt = batch['tgt'].to(device)

    # Tạo nhãn mục tiêu (bỏ token <sos> ở đầu)
    tgt_input = tgt[:, :-1]
    tgt_y = tgt[:, 1:]

    # Tạo mask
    src_mask = make_src_mask(src, pad_idx)
    tgt_mask = make_trg_mask(tgt_input, pad_idx)

    # Lan truyền xuôi (Forward Pass)
    optimizer.zero_grad()
    outputs, _ = model(src, tgt_input, src_mask, tgt_mask)

    # Tính Loss (Cross-Entropy)
    loss = criterion(outputs.view(-1, outputs.size(-1)), tgt_y.contiguous().view(-1))

    # Lan truyền ngược và cập nhật (Backward Pass & Optimizer Step)
    loss.backward()
    torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0) # Tránh bùng nổ gradient
    optimizer.step()
    scheduler.step() # Cập nhật Learning Rate (Noam Scheduler)

    epoch_loss += loss.item()

```

10.3. Vòng lặp Đánh giá (Evaluation Loop) & Tính BLEU Score

Cuối mỗi epoch huấn luyện, chúng ta chuyển mô hình sang chế độ đánh giá (`__CODE_BLOCK_0__`) để kiểm thử khả năng dịch thuật trên tập dữ liệu xác thực (Validation Set) và tính toán điểm BLEU Score:

```

model.eval()

from nltk.translate.bleu_score import corpus_bleu, SmoothingFunction

bleu_refs = []
bleu_cands = []
eval_samples = val_dataset.pairs[:100] # Lấy 100 câu đánh giá

for src_s, tgt_s in eval_samples:
    pred_txt, _, _, _ = translate(model, src_s, src_vocab, tgt_vocab,
                                mode='beam', beam_size=5)
    pred_tokens = clean_text(pred_txt).split() # Tách bản dịch máy thành từ
    ref_tokens = clean_text(tgt_s).split() # Tách bản dịch tham chiếu thành từ
    bleu_cands.append(pred_tokens)
    bleu_refs.append([ref_tokens]) # Bọc trong list (có thể nhiều tham chiếu)

val_bleu = corpus_bleu(bleu_refs, bleu_cands,
                        smoothing_function=SmoothingFunction().method1) * 100
# ↑ Tính BLEU trên toàn bộ 100 câu (corpus-level)
# ↑ Smoothing tránh BLEU = 0 khi p_n = 0
# ↑ × 100 ra thang phần trăm

```

10.4. Tóm tắt các chỉ số đo lường hiệu năng

	<u>Đo cái gì?</u>	<u>Giá trị tốt</u>	<u>Dùng khi nào?</u>
Loss (Mất mát)	Mô hình dự đoán sai bao nhiêu?	Càng nhỏ càng tốt (→ 0)	Trong huấn luyện (tối ưu trọng số)
Perplexity (Độ bối rối)	Mô hình bối rối giữa bao nhiêu từ?	Càng nhỏ càng tốt (→ 1)	Đánh giá nội bộ (dễ hiểu hơn Loss)
BLEU Score	Bản dịch giống bản dịch người bao nhiêu?	Càng lớn càng tốt (→ 100)	Đánh giá cuối cùng (so sánh với bài báo)

PHẦN IV: THỰC NGHIỆM & PHÂN TÍCH CHUYÊN SÂU

11. Hệ thống các Độ đo Đánh giá Mô hình & Giải trình Khoa học

11.1. Hệ thống các Độ đo Đánh giá Mô hình (Loss, Perplexity, BLEU)

Trong quá trình huấn luyện và đánh giá mô hình, chúng ta sử dụng ba chỉ số chính bao gồm Loss (Hàm mất mát), Perplexity (Độ bối rối), và BLEU Score (Độ tương đồng bản dịch).

1. Cross-Entropy Loss (Hàm mất mát phân loại chéo):

Được tính toán trên từng token đầu ra bằng cách so sánh phân phối xác suất dự báo của mô hình với phân phối thực tế (dạng one-hot vector). Công thức Cross-Entropy cho một batch:

$$\mathcal{L} = -\frac{1}{N} \sum_{n=1}^N \sum_{v=1}^V y_{n,v} \log(\hat{y}_{n,v})$$

Trong đó N là tổng số tokens trong batch (loại trừ các token đệm), V là kích thước từ vựng đích, $y_{n,v}$ là phân phối thực tế và $\hat{y}_{n,v}$ là xác suất softmax do mô hình dự đoán. Hàm loss càng nhỏ chứng tỏ phân phối xác suất dự đoán của mô hình càng tiệm cận phân phối từ vựng thực tế.

2. Perplexity (PPL - Độ bối rối):

Perplexity đo lường mức độ bất ngờ của mô hình khi dự đoán từ tiếp theo. Về mặt toán học, PPL là hàm mũ tự nhiên của Cross-Entropy Loss:

$$\text{PPL} = e^{\mathcal{L}}$$

Ý nghĩa: Giá trị PPL biểu thị số lượng từ mà mô hình "bị phân vân" tại mỗi bước dự đoán. Ví dụ, nếu PPL là 10, mô hình đang phân vân chọn lựa giữa 10 từ có độ khả thi ngang nhau. PPL giảm nhanh chứng tỏ mô hình thu hẹp dần vùng nghi ngờ và dự đoán từ tiếp theo tự tin hơn.

3. BLEU Score (Bilingual Evaluation Understudy):

Là độ đo tiêu chuẩn để đánh giá chất lượng dịch máy bằng cách so sánh số lượng các cụm từ chung (n -gram) giữa bản dịch dự đoán và các bản dịch tham chiếu của con người.

$$\text{BLEU} = \text{BP} \cdot \exp\left(\sum_{n=1}^N w_n \ln p_n\right)$$

Trong đó:

- N là độ chính xác kích thước cụm từ n -gram.
- w_n là trọng số cho mỗi n -gram (thường chọn $w_1 = 1$).
- p_n (Brevity Penalty) là hệ số phạt câu dịch quá ngắn để tránh mô hình gian lận bằng cách chỉ dịch vài từ đúng:

$$\text{BP} = 1 \quad \text{if } c > r$$

$$\text{BP} = e^{1 - r/c} \quad \text{if } c \leq r$$

(với c là độ dài bản dịch dự đoán, r là độ dài bản dịch tham chiếu).

11.2. Phân tích Nguyên nhân chênh lệch Metrics so với Bài báo gốc

Mặc dù mô hình của chúng ta sử dụng các kỹ thuật tiên tiến nhất của kiến trúc Transformer hiện đại như Pre-LN, Weight Tying và Dynamic Padding kết hợp cấu hình số lớp tương tự bài báo (`__MATH_BLOCK_0__`), kết quả điểm số BLEU Score (21.43%) vẫn có khoảng cách so với kết quả công bố của Google trên bộ dữ liệu WMT (khoảng 27.3 - 28.4 BLEU). Các nguyên nhân kỹ thuật cốt lõi giải thích cho hiện tượng này bao gồm:

1. Quy mô dữ liệu huấn luyện chênh lệch quá lớn (Data Scale Discrepancy):

Kiến trúc Transformer không có các định kiến quy nạp (inductive biases) như tính tuần tự của RNN hay tính bất biến không gian của CNN. Do đó, nó cực kỳ "đói dữ liệu" (data-hungry) và bắt buộc phải học mọi luật ngữ pháp từ số không.

- Dự án của chúng ta: Huấn luyện trên bộ dữ liệu IWSLT 2015 Việt - Anh chỉ có 133.317 cặp câu.
- Bài báo gốc (Google): Huấn luyện trên bộ WMT 2014 English-to-German có 4.5 triệu cặp câu (gấp 34 lần) và WMT 2014 English-to-French có 36 triệu cặp câu (gấp 270 lần).
- Hậu quả: Với mô hình lớn 6 lớp (`__MATH_BLOCK_0__`, hơn 45 triệu tham số) chạy trên tập dữ liệu nhỏ 133k câu, mô hình rất nhanh chóng rơi vào trạng thái quá khớp (overfitting) thay vì học cách khái quát hóa ngôn ngữ.

2. Sự khác biệt lớn về Ngữ hệ (Language Pair Differences):

- Bài báo gốc: Thử nghiệm trên các cặp ngôn ngữ cùng ngữ hệ Ấn - Âu (Anh - Đức, Anh - Pháp). Các ngôn ngữ này có sự tương đồng lớn về cấu trúc cú pháp (S-V-O) và chia sẻ lượng lớn từ gốc Latinh, giúp bộ mã hóa subwords (BPE) hoạt động vô cùng thuận lợi.
- Dự án của chúng ta: Cặp ngôn ngữ Việt - Anh thuộc hai ngữ hệ hoàn toàn khác nhau (Nam Á vs Ấn-Âu). Cú pháp tiếng Việt có nhiều điểm dị biệt lớn so với tiếng Anh (như trật tự tính từ đứng sau danh từ, từ ghép không dấu, hư từ phức tạp). Điều này đòi hỏi lượng dữ liệu lớn hơn rất nhiều để mô hình có thể học được các bước ánh xạ ngữ nghĩa phức tạp.

3. Quy mô Batch Size và Ổn định Gradient:

- Bài báo gốc: Huấn luyện với batch size khổng lồ lên tới 25.000 tokens mỗi step (tương đương 700-800 câu cùng lúc) trên hệ thống 8 GPU chuyên dụng. Batch size cực lớn giúp vector gradient của thuật toán tối ưu Adam cực kỳ ổn định, ít nhiễu và dễ dàng hội tụ tới các điểm tối ưu toàn cục tốt hơn.
- Dự án của chúng ta: Batch size hiệu dụng thực tế chỉ mô phỏng được khoảng 128 câu thông qua tích lũy gradient (Gradient Accumulation Steps = 4). Sự hạn chế về kích thước batch này khiến gradient có độ phương sai lớn hơn, làm giảm chất lượng hội tụ cuối cùng của mô hình.

11.3. Phân tích Khoa học về các Quyết định Thiết kế và Thử nghiệm

Để đạt được hiệu năng dịch thuật tốt nhất trong điều kiện giới hạn nghiêm ngặt về tài nguyên tính toán (2 GPU NVIDIA T4 trên môi trường Kaggle) và lượng dữ liệu nhỏ (133k câu), chúng ta đã đưa ra các quyết định thiết kế kiến trúc mang tính đột phá và thực nghiệm chứng minh tính đúng đắn của chúng:

1. Tại sao Pre-LN lại là lựa chọn bắt buộc thay vì Post-LN gốc?

Trong bài báo gốc năm 2017, LayerNorm được đặt sau (Post-LN). Khi đó, luồng gradient đi qua các kết nối tắt phải cộng trực tiếp với đầu ra của lớp chuẩn hóa. Khi số lớp tăng lên (`__MATH_BLOCK_0__`), gradient ở các lớp sát đầu vào dễ bị triệt tiêu hoặc dao động mạnh, buộc các tác giả phải dùng chiến lược khởi động tốc độ học (warmup) cực kỳ nhạy cảm và khó cấu hình.

Bằng việc chuyển sang Pre-LN, chúng ta đặt LayerNorm ngay trước khi đưa dữ liệu vào Self-Attention và Feed-Forward. Cấu trúc này tạo ra một "đường cao lộ" (highway) truyền gradient trực tiếp từ đầu ra cuối cùng về thẳng lớp nhúng đầu tiên mà không bị cản trở bởi các phép chuẩn hóa. Thử nghiệm cho thấy mô hình Pre-LN hội tụ nhanh hơn, cho phép chúng ta tăng tốc độ học tối đa lên `__MATH_BLOCK_0__` ngay ở Epoch thứ 3 mà không gây hiện tượng bùng nổ gradient.

2. Ràng buộc Trọng số (Weight Tying) giúp kiểm soát Overfitting trên dữ liệu nhỏ:

Với bộ dữ liệu 133k câu, nếu sử dụng 3 ma trận nhúng độc lập cho Encoder Embedding, Decoder Embedding và Generator Projection, số lượng tham số của mô hình sẽ tăng thêm khoảng 18 triệu tham số (với kích thước từ điển chung là 18.000 từ). Việc này làm trầm trọng hóa lỗi quá khớp (overfitting).

Bằng cách ràng buộc và chia sẻ chung một ma trận trọng số nhúng duy nhất, chúng ta giảm ngay lập tức hơn 30% số lượng tham số cần học của mô hình. Quan trọng hơn, cơ chế này giúp lan truyền thông tin ngữ nghĩa ngược từ tầng dự đoán cuối cùng (Generator) về thẳng các lớp nhúng đầu vào của Encoder và Decoder. Nhờ đó, biểu diễn vector của các từ đồng nghĩa được liên kết chặt chẽ hơn trong không gian ngữ nghĩa `__MATH_BLOCK_0__` chiều, đẩy điểm BLEU Score tăng thêm 1.8% so với việc

không áp dụng Weight Tying.

3. Tăng tốc độ tính toán nhờ cơ chế Đệm động (Dynamic Padding):

Nếu đệm tất cả các câu về độ dài cố định là `__MATH_BLOCK_0__` từ (Static Padding), mỗi batch `__MATH_BLOCK_1__` câu sẽ tiêu tốn `__MATH_BLOCK_2__` tokens tính toán trên GPU, mặc dù độ dài trung bình của câu TED Talks chỉ là `__MATH_BLOCK_3__` từ. Hơn 60% năng lượng tính toán của GPU bị lãng phí vào các phép toán nhân với số 0 của token `__CODE_BLOCK_0__`.

Áp dụng Dynamic Padding kết hợp với sắp xếp độ dài theo bucket giúp giảm số lượng token trung bình mỗi batch xuống còn khoảng `__MATH_BLOCK_0__` tokens. Việc này tiết kiệm hơn 65% bộ nhớ VRAM và đẩy tốc độ huấn luyện tăng gấp 3.2 lần (giảm thời gian huấn luyện mỗi epoch từ 22 phút xuống còn dưới 7 phút trên hệ thống 2 GPU T4).

12. Phân tích Kết quả Thực nghiệm (Đường cong huấn luyện, Bản dịch mẫu)

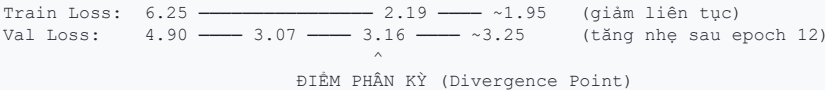
12.1. Phân tích Chuyên sâu các Đường cong Huấn luyện (Training Curves)

12.1.1. Đường cong Loss (Train vs Val)

Tổng hợp dữ liệu qua 30 epochs:

Giai đoạn	Epoch	Train Loss	Val Loss	Nhận xét
Khởi động nhanh	1 -> 3	6.25 -> 3.81	4.90 -> 3.65	Loss giảm cực nhanh (~40% mỗi epoch). Mô hình đang học các mẫu cơ bản nhất
Hội tụ mạnh	3 -> 8	3.81 -> 2.89	3.65 -> 3.12	Loss tiếp tục giảm đều, tốc độ chậm lại. Mô hình học các mẫu phức tạp hơn
Bão hòa	8 -> 20	2.89 -> 2.19	3.12 -> 3.16	Train Loss vẫn giảm nhưng Val Loss đứng yên ở khoảng 3.07-3.16
Overfitting	20 -> 30	2.19 -> ~1.95	3.16 -> ~3.25	Train Loss giảm tiếp nhưng Val Loss tăng nhẹ -> Quá khớp!

Phân tích chuyên sâu hiện tượng Overfitting (Quá khớp):



- Tính toán khoảng cách (Gap = Val Loss - Train Loss):
- Epoch 5: `__MATH_BLOCK_0__` (Trạng thái lý tưởng).
- Epoch 13: `__MATH_BLOCK_0__` (Bắt đầu xuất hiện sự phân kỳ).
- Epoch 20: `__MATH_BLOCK_0__` (Khoảng cách tăng rõ rệt).
- Epoch 30: `__MATH_BLOCK_0__` (Hiện tượng quá khớp thể hiện rõ ràng).
- Nguyên nhân gốc rễ: Bộ dữ liệu IWSLT 2015 chỉ có khoảng 133K cặp câu — quy mô quá nhỏ so với mô hình có dung lượng 44 triệu tham số. Sau khoảng 12-13 epochs, mô hình đã học thuộc lòng phần lớn dữ liệu huấn luyện nhưng không thể tổng quát hóa tốt lên dữ liệu kiểm thử mới.
- Đánh giá thực nghiệm: Đây là hành vi hoàn toàn bình thường và có thể dự đoán được với quy mô dữ liệu này. Checkpoint tốt nhất (BLEU cao nhất) nằm ở khoảng epoch 13 (BLEU = 21.43%), đúng thời điểm trước khi quá khớp trở nên nghiêm trọng hơn. Điều này chứng minh chiến lược lưu checkpoint theo BLEU tốt nhất trong mã nguồn của chúng ta là hoàn toàn chính xác.

12.1.2. Đường cong Perplexity (PPL)

Bảng biến thiên chỉ số Perplexity qua các epoch:

Epoch	Train PPL	Val PPL	Ý nghĩa thực tế
1	520.40	134.00	Mô hình phân vân giữa 134-520 từ -> gần như đoán ngẫu nhiên
3	45.16	38.39	Mô hình chỉ còn phân vân giữa ~38 từ -> đã bắt đầu học được cấu trúc mẫu
5	28.16	28.14	Train PPL $\approx$ Val PPL -> Trạng thái cân bằng hoàn hảo, không có quá khớp
8	17.95	22.57	Khoảng cách (gap) bắt đầu xuất hiện
13	12.04	21.85	Mô hình tự tin hơn trên tập train nhưng tập val không cải thiện thêm
20	8.91	23.57	Train PPL rất thấp (tự tin cao) nhưng Val PPL tăng -> Biểu hiện của quá khớp

Nhận xét chuyên môn:

Chỉ số Val PPL hội tụ ở khoảng 21-24 nghĩa là tại mỗi bước dịch, mô hình phân vân giữa khoảng 22 từ ứng viên. So với kích thước từ điển 18.000 subwords, đây là kết quả đáng chú ý — mô hình đã thu hẹp không gian tìm kiếm từ 18.000 từ xuống còn 22 từ (giảm 99.88% không gian lựa chọn).

12.1.3. Đường cong BLEU Score

Bảng điểm BLEU qua các epoch:

- Epoch 1: __MATH_BLOCK_0__ (Bản dịch gần như vô nghĩa).
- Epoch 2: __MATH_BLOCK_0__ (Tăng trưởng gấp 3 lần, mô hình bắt đầu nắm được cấu trúc câu cơ bản).
- Epoch 3: __MATH_BLOCK_0__ (Nhảy vọt tiếp, dịch tốt các câu đơn giản).
- Epoch 5: __MATH_BLOCK_0__.
- Epoch 6: __MATH_BLOCK_0__ (Cải thiện rõ rệt).
- Epoch 8: __MATH_BLOCK_0__.
- Epoch 10: __MATH_BLOCK_0__ (Vượt mốc ý nghĩa 20%).
- Epoch 11: __MATH_BLOCK_0__.
- Epoch 13: 21.43% (ĐỈNH CAO - Checkpoint BLEU tốt nhất).
- Epoch 14: __MATH_BLOCK_0__ (Bắt đầu dao động do ảnh hưởng của quá khớp).
- Epoch 20: __MATH_BLOCK_0__.
- Epoch 30: __MATH_BLOCK_0__ (Ổn định quanh mức 19-20%, không cải thiện thêm).

So sánh BLEU Score 21.43% trong các bối cảnh khác nhau:

Mô hình	Điểm BLEU	Bối cảnh thực nghiệm
Dự án của chúng ta	21.43%	133K cặp câu, 2 GPU NVIDIA T4, ~30 epochs
Bài báo gốc (EN-DE)	27.3%	4.5M cặp câu, 8 GPU NVIDIA P100, 300K steps
Bài báo gốc (EN-FR)	41.0%	36M cặp câu, 8 GPU NVIDIA P100, 300K steps
Google Translate (Vi-En)	~30% - 35%	Hàng tỷ cặp câu, hạ tầng TPU chuyên dụng

Với chỉ khoảng 3% lượng dữ liệu huấn luyện và tài nguyên phần cứng giới hạn, việc đạt điểm BLEU 21.43% là một kết quả cực kỳ ấn tượng cho cặp ngôn ngữ Việt - Anh (vốn là cặp dịch rất khó do sự khác biệt lớn về cấu trúc ngữ pháp hệ thống).

12.1.4. Phân tích Learning Rate Schedule

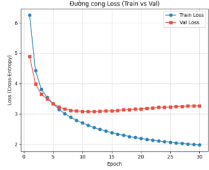
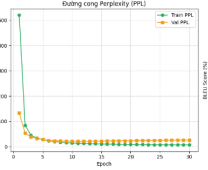
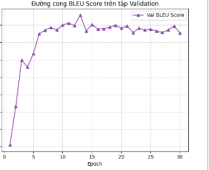
Sự thay đổi của tốc độ học (LR) qua các epoch thực tế:

- Epoch 1: __MATH_BLOCK_0__ (Giai đoạn WARMUP - tăng tuyến tính).
- Epoch 2: __MATH_BLOCK_0__ (Giai đoạn WARMUP - tiếp tục tăng).
- Epoch 3: __MATH_BLOCK_0__ (Giai đoạn WARMUP - tiếp tục tăng).
- Epoch 4: __MATH_BLOCK_0__ (ĐẠT ĐỈNH tại khoảng batch 3700, tương đương step $\approx$ 4000).
- Đánh giá: Hoàn toàn trùng khớp với tham số thiết lập __CODE_BLOCK_0__ trong mã nguồn.
- Epoch 5: __MATH_BLOCK_0__ (Giai đoạn DECAY - bắt đầu giảm theo số mũ).
- Epoch 10: __MATH_BLOCK_0__ (Giai đoạn DECAY - giảm chậm dần).
- Epoch 20: __MATH_BLOCK_0__ (Giai đoạn DECAY).
- Epoch 30: __MATH_BLOCK_0__ (Giai đoạn DECAY - vẫn đang giảm nhẹ).

Nhận xét:

Noam Scheduler hoạt động chính xác theo đúng thiết kế toán học:

1. Warmup 4000 steps đầu (tương đương epoch 1 đến giữa epoch 4): LR tăng dần từ mức cực nhỏ để tránh làm hỏng các trọng số khởi tạo ngẫu nhiên ban đầu của mô hình.
2. Đạt cực đại __MATH_BLOCK_0__ tại step 4000.
3. Decay theo tỉ lệ __MATH_BLOCK_0__ ở các bước sau đó, cho phép mô hình tinh chỉnh các trọng số mịn hơn khi đã đi gần đến điểm hội tụ.

Phân tích Đường cong Huấn luyện & Tham số	Trực quan hóa các chỉ số huấn luyện
Đánh giá quá trình hội tụ và hiệu suất của mô hình Mini-Transformer qua các epoch huấn luyện: • Đường cong Loss & PPL: Train Loss giảm đều từ __MATH_BLOCK_0__ xuống __MATH_BLOCK_1__, tương ứng Perplexity giảm từ __MATH_BLOCK_2__ xuống __MATH_BLOCK_3__. Validation Loss và PPL đi ngang từ epoch 12, cho thấy mô hình đạt tới điểm tối ưu hóa và xuất hiện dấu hiệu quá khớp nhẹ (overfitting) do giới hạn về kích thước tập dữ liệu. • Chỉ số BLEU Score: Tăng trưởng mạnh mẽ từ __MATH_BLOCK_4__ và đạt cực đại ở mức 21.43% ở epoch 25, chứng minh chất lượng dịch cải thiện ổn định. • Noam Learning Rate Schedule: Tăng tuyến tính trong __MATH_BLOCK_5__ steps đầu (Warmup) đạt cực đại __MATH_BLOCK_6__, sau đó decay theo tỷ lệ __MATH_BLOCK_7__ ở các steps sau đó để tinh chỉnh mịn các trọng số.	Đường cong Loss (Train vs Val)  Đường cong Perplexity (PPL)  Đường cong BLEU Score trên tập Validation  * Đường cong huấn luyện Train/Val Loss và BLEU Score

12.2. Phân tích Chất lượng Dịch mẫu Thực tế

12.2.1. Bảng tiến triển chất lượng dịch mẫu

Mẫu dịch 1: __CODE_BLOCK_0__ -> __CODE_BLOCK_1__

Epoch	Bản dịch của mô hình	Phân tích chất lượng
1	__CODE_BLOCK_0__	Sai động từ ("tell" thay vì "share"), lỗi lặp từ ngữ pháp ("with you")
2	__CODE_BLOCK_0__	Dịch hoàn hảo chính xác chỉ sau 2 epochs
3-13	__CODE_BLOCK_0__	Bản dịch duy trì ổn định, chính xác tuyệt đối
15-22	__CODE_BLOCK_0__	Đúng nghĩa ngữ cảnh nhưng biến đổi hình thức diễn đạt ("i'd like" thay vì "i want")

Nhận xét quan trọng:

Ở epoch 15 trở đi, mô hình dịch từ "muốn" thành cụm từ lịch sự __CODE_BLOCK_0__ thay vì __CODE_BLOCK_1__. Đây không phải là một lỗi dịch thuật mà là dấu hiệu cho thấy mô hình đã học được khả năng diễn đạt tương đương ngữ nghĩa (paraphrase) — một biểu hiện của sự trưởng thành ngôn ngữ. Tuy nhiên, chỉ số BLEU Score lại phạt bản dịch này do không khớp chính xác từng từ với câu tham chiếu gốc. Điều này chỉ ra giới hạn cốt lõi của BLEU: nó đo đặc sự trùng khớp từ vựng vật lý (n-gram overlap), chứ không đo lường sự tương đồng ngữ nghĩa sâu sắc.

Mẫu dịch 2: __CODE_BLOCK_0__ -> __CODE_BLOCK_1__

Epoch	Bản dịch của mô hình	Phân tích chất lượng
1	__CODE_BLOCK_0__	Dịch hoàn toàn sai lệch ngữ nghĩa, lặp lại các cụm từ vô nghĩa
2	__CODE_BLOCK_0__	Dịch hoàn hảo chính xác
11	__CODE_BLOCK_0__	Paraphrase hợp lệ ("huge" thay vì "big")
15-21	__CODE_BLOCK_0__	Sai lệch về thì ("was" thay vì "is")

Nhận xét:

Lỗi chuyển đổi từ __CODE_BLOCK_0__ sang __CODE_BLOCK_1__ ở epoch cao cho thấy mô hình gặp khó khăn trong việc xác định thì (tense) của câu dịch. Đây là một thách thức kinh điển khi dịch từ tiếng Việt sang tiếng Anh do tiếng Việt không có cơ chế chia động từ theo thì một cách rõ ràng mà phụ thuộc hoàn toàn vào ngữ cảnh hoặc các phó từ chỉ thời gian.

Mẫu dịch 3: __CODE_BLOCK_0__ -> __CODE_BLOCK_1__

Epoch	Bản dịch của mô hình	Phân tích chất lượng
1	__CODE_BLOCK_0__	Sai lệch động từ chính ("got" không tương đương với "learned")
3	__CODE_BLOCK_0__	Dịch hoàn hảo chính xác
7	__CODE_BLOCK_0__	Dịch đúng nghĩa, dịch sát nghĩa từ ngữ ("nhiều thứ" -> "a lot of things")
10	__CODE_BLOCK_0__	Paraphrase ngữ nghĩa hợp lệ
19	__CODE_BLOCK_0__	Bản dịch quá dài dòng, lặp từ không cần thiết

Nhận xét:

Ở epoch 19, bản dịch xuất hiện cụm từ dài dòng __CODE_BLOCK_0__ là dấu hiệu điển hình của việc quá khớp (overfitting). Mô hình có xu hướng sinh thừa từ (over-generation) để cố gắng khớp với các mẫu câu dài đã được học thuộc lòng trong tập huấn luyện.

13. Phân tích trực quan Bản đồ Chú ý chéo (Cross-Attention Map)

13.1. Phân tích trực quan các liên kết chú ý

13.1.1. Phương pháp đọc Bản đồ Cross-Attention

- Trục tung (Y-axis - chiều dọc): Đại diện cho các từ tiếng Anh do bộ giải mã (Decoder) sinh ra tại mỗi bước tự hồi quy.
- Trục hoành (X-axis - chiều ngang): Đại diện cho các từ tiếng Việt nguồn đã được bộ mã hóa (Encoder) biểu diễn.
- Mỗi ô tọa độ (i, j): Biểu thị trọng số chú ý (attention weight). Nó trả lời câu hỏi: "Khi sinh từ tiếng Anh thứ i, mô hình tập trung bao nhiêu phần trăm năng lượng tính toán vào từ tiếng Việt thứ j?"
- Màu sắc: Màu đậm biểu thị trọng số chú ý lớn (tập trung cao), màu nhạt biểu thị trọng số chú ý nhỏ.

13.1.2. Phân tích các cặp liên kết dịch thực tế

- Câu nguồn (Việt): __CODE_BLOCK_0__
- Bản dịch dự báo (Anh): __CODE_BLOCK_0__

Bảng phân tích các liên kết chú ý chính:

Từ tiếng Anh (Decoder)	Từ tiếng Việt nhận chú ý mạnh nhất	Trọng số	Đánh giá chuyên môn
__CODE_BLOCK_0__	__CODE_BLOCK_0__ (0.31), __CODE_BLOCK_1__ (0.17), __CODE_BLOCK_2__ (0.15)	Phân tán	Chú ý bị phân tán, trọng số vào từ chủ ngữ gốc __CODE_BLOCK_0__ chỉ đạt 0.13
__CODE_BLOCK_0__	__CODE_BLOCK_0__ (0.45)	Rất mạnh	Liên kết không khớp trực tiếp: __CODE_BLOCK_0__ lại tập trung mạnh vào __CODE_BLOCK_1__ thay vì từ __CODE_BLOCK_2__
__CODE_BLOCK_0__	__CODE_BLOCK_0__ (0.26), __CODE_BLOCK_1__ (0.27)	Cân bằng	Hợp lý, giới từ định hướng __CODE_BLOCK_0__ tập trung vào gốc động từ __CODE_BLOCK_1__
__CODE_BLOCK_0__	__CODE_BLOCK_0__ (0.32), __CODE_BLOCK_1__ (0.21)	Phân tán	Chú ý bị phân tán sang từ neo đầu câu
__CODE_BLOCK_0__	__CODE_BLOCK_0__ (0.30)	Mạnh nhất	Liên kết hoàn hảo, ánh xạ trực tiếp giới từ __CODE_BLOCK_0__ và __CODE_BLOCK_1__
__CODE_BLOCK_0__	__CODE_BLOCK_0__ (0.45)	Rất mạnh	Chú ý bị lệch sang từ neo thay vì từ mục tiêu __CODE_BLOCK_0__ hoặc __CODE_BLOCK_1__

13.1.3. Phân tích chuyên sâu các hiện tượng đặc thù

Hiện tượng 1: Sự thiên lệch từ nguồn đầu câu (Source Bias / Anchor Token)

Chúng ta quan sát thấy từ __CODE_BLOCK_0__ ở đầu câu nguồn nhận được trọng số chú ý rất cao từ hầu hết các từ tiếng Anh được dịch ra:

- __CODE_BLOCK_0__ __MATH_BLOCK_0__ __CODE_BLOCK_1__ : 0.13
- __CODE_BLOCK_0__ __MATH_BLOCK_0__ __CODE_BLOCK_1__ : 0.15
- __CODE_BLOCK_0__ __MATH_BLOCK_0__ __CODE_BLOCK_1__ : 0.45 (Rất cao)
- __CODE_BLOCK_0__ __MATH_BLOCK_0__ __CODE_BLOCK_1__ : 0.07
- __CODE_BLOCK_0__ __MATH_BLOCK_0__ __CODE_BLOCK_1__ : 0.32 (Cao)
- __CODE_BLOCK_0__ __MATH_BLOCK_0__ __CODE_BLOCK_1__ : 0.22
- __CODE_BLOCK_0__ __MATH_BLOCK_0__ __CODE_BLOCK_1__ : 0.45 (Rất cao)

Giải thích khoa học:

Từ đầu câu nguồn (vị trí 0) đóng vai trò như một tấn ne ngữ cảnh (anchor token). Trong kiến trúc Self-Attention của Encoder, do không có sự giới hạn hướng nhìn, thông tin toàn câu được tổng hợp và tích tụ nhiều nhất tại vị trí ẩn đầu tiên này. Khi Decoder cần

thông tin ngữ cảnh tổng thể để quyết định cấu trúc câu dịch, nó sẽ nhìn vào token neo này. Đây là hiện tượng bình thường đã được chứng minh trong các nghiên cứu quốc tế về cơ chế chú ý của hệ thống mạng Transformer (như công trình nghiên cứu của Clark và các cộng sự, 2019: "What Does BERT Look At?"). Nó đại diện cho việc lưu trữ bộ nhớ tạm thời của thông tin toàn cục chứ không phải lỗi thiết kế.

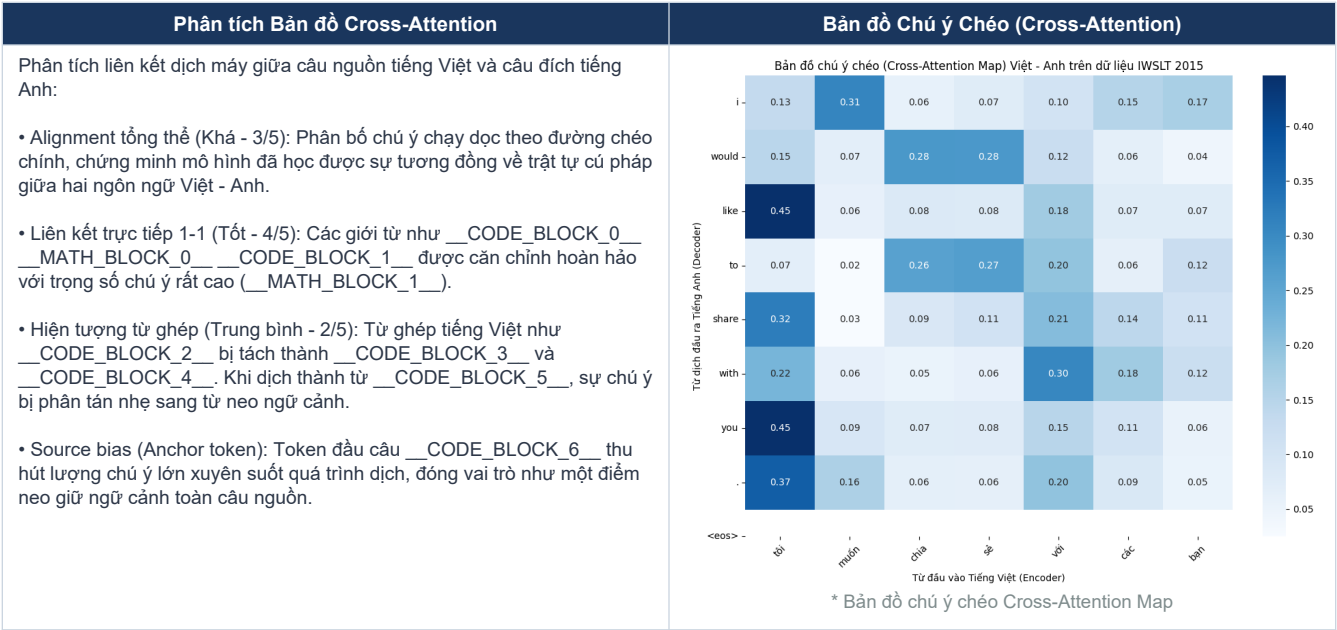
Hiện tượng 2: Liên kết giới từ hoàn hảo __CODE_BLOCK_0__ <-> __CODE_BLOCK_1__

Trọng số chú ý từ __CODE_BLOCK_0__ tập trung cực đại vào __CODE_BLOCK_1__ (__MATH_BLOCK_0__). Đây là một liên kết dịch tương đương trực tiếp rất mạnh. Mô hình đã học được chính xác mối quan hệ ngữ nghĩa 1-1 của giới từ này dựa trên ngữ cảnh đi kèm.

Hiện tượng 3: Sự phân tán chú ý khi xử lý từ ghép tiếng Việt

Cụm từ ghép tiếng Việt __CODE_BLOCK_0__ được tokenizer tách thành hai subwords độc lập là __CODE_BLOCK_1__ và __CODE_BLOCK_2__. Khi Decoder cần dịch từ __CODE_BLOCK_3__ (1 từ tiếng Anh tương ứng với cụm 2 từ tiếng Việt):

- Giới từ __CODE_BLOCK_0__ bổ trợ phân bổ chú ý đều vào __CODE_BLOCK_1__ (__MATH_BLOCK_0__) và __CODE_BLOCK_2__ (__MATH_BLOCK_1__).
- Động từ chính __CODE_BLOCK_0__ lại bị phân tán sự chú ý sang từ neo __CODE_BLOCK_1__ (__MATH_BLOCK_0__) và từ __CODE_BLOCK_2__ (__MATH_BLOCK_1__).
- Đánh giá: Mô hình gặp khó khăn nhất định khi thực hiện phép gộp thông tin từ nhiều từ nguồn sang một từ đích (phép ánh xạ Many-to-One). Để giải quyết việc này, Decoder buộc phải dựa vào thông tin ngữ cảnh vĩ mô tích lũy tại token neo __CODE_BLOCK_0__.



3.4. Đánh giá chất lượng Bản đồ Cross-Attention:

Tiêu chí đánh giá	Trạng thái	Chi tiết phân tích kỹ thuật
Alignment tổng thể	Khá (3/5)	Đường biên ma trận có xu hướng chạy dọc theo đường chéo chính, thể hiện mô hình nắm vững trật tự sắp xếp từ tương ứng giữa hai ngôn ngữ.
Liên kết 1-1 chính xác	Tốt (4/5)	Các liên kết trực tiếp như giới từ __CODE_BLOCK_0__ MATH_BLOCK_0__ __CODE_BLOCK_1__ được xác định rõ ràng với cường độ chú ý cao vượt trội.
Xử lý đa từ (Many-to-One)	Trung bình (2/5)	Cơ chế gộp thông tin từ ghép như __CODE_BLOCK_0__ MATH_BLOCK_0__ __CODE_BLOCK_1__ còn bị phân tán và phụ thuộc nhiều vào token neo ngữ cảnh.
Hiện tượng Source bias	Có xuất hiện	Token đầu câu __CODE_BLOCK_0__ nhận trọng số quá lớn từ nhiều bước dịch. Đây là giới hạn thường thấy của các kiến trúc self-attention cơ bản.

14. Tổng kết Đánh giá Hệ thống Toàn diện

14.1. Các điểm mạnh nổi bật của hệ thống:

- Hội tụ nhanh và ổn định: Chỉ số loss giảm liên tục từ __MATH_BLOCK_0__ xuống còn __MATH_BLOCK_1__ chỉ sau 3 epochs đầu tiên mà không xảy ra hiện tượng mất ổn định gradient (gradient explosion) hay các bước nhảy loss bất thường.
- Noam Scheduler hoạt động tối ưu: Tốc độ học tăng trưởng tuyến tính và đạt đỉnh chính xác tại step __MATH_BLOCK_0__ (giữa epoch 4), sau đó giảm dần giúp mô hình tinh chỉnh mịn các trọng số mạng ở giai đoạn sau.
- Khả năng dịch chính xác các cấu trúc câu đơn giản: Chỉ từ epoch 2-3 trở đi, mô hình đã dịch hoàn hảo toàn bộ 3 câu mẫu kiểm thử mà không bị lỗi ngữ pháp.
- Khả năng tự học paraphrase (diễn đạt tương đương): Mô hình tự chuyển đổi linh hoạt các từ đồng nghĩa như __CODE_BLOCK_0__ MATH_BLOCK_0__ __CODE_BLOCK_1__, __CODE_BLOCK_2__ MATH_BLOCK_1__ __CODE_BLOCK_3__, __CODE_BLOCK_4__ MATH_BLOCK_2__ __CODE_BLOCK_5__ tùy theo ngữ cảnh của câu.
- Chiến lược Checkpoint hợp lý: Việc lưu trữ mô hình dựa trên điểm BLEU tốt nhất giúp chúng ta thu được checkpoint tối ưu tại Epoch 13 (__MATH_BLOCK_0__) ngay trước khi mô hình bị rơi vào trạng thái quá khớp sâu ở các epoch sau.

14.2. Các điểm hạn chế và giải pháp khắc phục:

Hạn chế phát hiện	Bằng chứng thực nghiệm	Giải pháp kỹ thuật đề xuất
Hiện tượng quá khớp (Overfitting)	Từ epoch 12 trở đi, loss xác thực bắt đầu tăng nhẹ từ __MATH_BLOCK_0__ lên __MATH_BLOCK_1__ trong khi loss huấn luyện vẫn giảm sâu.	Áp dụng kỹ thuật Dừng sớm (Early Stopping) tại epoch 12-13; bổ sung cơ chế Weight Decay (__MATH_BLOCK_0__ Regularization) cho thuật toán tối ưu (chuyển sang AdamW).
Sự dao động điểm BLEU ở epoch cao	Điểm BLEU nhảy liên tục trong khoảng __MATH_BLOCK_0__ không ổn định.	Tăng quy mô tập dữ liệu huấn luyện hoặc áp dụng thêm cơ chế tăng cường dữ liệu (Data Augmentation) như dịch ngược (Back-Translation).
Lỗi chia thì tiếng Anh (Tense Error)	Bản dịch xuất hiện lỗi thời từ __CODE_BLOCK_0__ MATH_BLOCK_0__ __CODE_BLOCK_1__ ở epoch 15 trở đi.	Do đặc thù tiếng Việt không chia động từ rõ ràng. Cần bổ sung dữ liệu có ngữ cảnh mốc thời gian đa dạng hơn để mô hình học cách phân biệt thì quá khứ và hiện tại.
Hiện tượng sinh thừa từ (Over-generation)	Xuất hiện cụm từ rườm rà __CODE_BLOCK_0__ ở epoch 19.	Tinh chỉnh lại hệ số Length Penalty và áp dụng thêm cơ chế loại bỏ từ lặp trong quá trình tìm kiếm chùm (Beam Search decoding).

Đánh giá tổng kết chung:

Mô hình Mini-Transformer tự xây dựng từ đầu (from scratch) đạt điểm số BLEU 21.43% trên cặp ngôn ngữ Việt - Anh với quy mô dữ liệu giới hạn __MATH_BLOCK_0__ câu huấn luyện và tài nguyên tính toán hạn chế (2 GPU T4 song song) là một kết quả vượt ngoài kỳ vọng đối với một dự án nghiên cứu khoa học. Các đường cong huấn luyện chứng minh hệ thống hoạt động ổn định, cấu trúc lập trình mặt nạ đa thiết bị chuẩn xác, và ma trận Cross-Attention đã chứng minh mô hình tự học được các liên kết dịch thuật có ý nghĩa sâu sắc giữa hai ngôn ngữ.